GPU_Project_Image_Filtering_Using_FourierTransform_YanboZhu

1 Setup

1.1 install dependency

```
sudo apt update
sudo apt install -y build-essential cmake git unzip pkg-config
sudo apt install -y libjpeg-dev libpng-dev libtiff-dev ffmpeg
sudo apt install -y libavcodec-dev libavformat-dev libswscale-dev libv4l-dev
sudo apt install -y libxvidcore-dev libx264-dev libx265-dev
sudo apt install -y libgtk-3-dev libcanberra-gtk3-dev
sudo apt install -y libatlas-base-dev gfortran
sudo apt install -y python3-dev python3-pip
pip3 install numpy



# 下面的不用
sudo apt update && sudo apt install -y \
    build-essential \
    cmake \
    git \
    pkg-config \
    libgtk-3-dev \
    libavcodec-dev \
    libavformat-dev \
    libswscale-dev \
    libv4l-dev \
    libxvidcore-dev \
    libx264-dev \
    libjpeg-dev \
    libpng-dev \
    libtiff-dev \
    libwebp-dev \ZX
    libopenexr-dev \
    libgstreamer1.0-dev \
    libgstreamer-plugins-base1.0-dev \
    libatlas-base-dev \
    libtbb2 \
    libtbb-dev \
    libdc1394-22-dev \
    liblapacke-dev \
    gfortran \
    python3-dev \
    python3-numpy \
    ffmpeg
```

1.2 Install OpenCV (Please skip it)

```
mkdir -r opencv/source

git clone https://github.com/opencv/opencv.git
git checkout 4.13.0

mkdir opencv_contrib
cd opencv_contrib
git clone https://github.com/opencv/opencv_contrib.git
git checkout 4.13.0
```

1.2.1 compile opencv with cmake

```
# ~/opencv
mkdir build
```

```
.
└── opencv
    ├── build
    └── source
        ├── opencv
        └── opencv_contrib
```

```
cd build

cmake \
-D CMAKE_BUILD_TYPE=RELEASE \
-D CMAKE_INSTALL_PREFIX=/usr/local \
-D OPENCV_EXTRA_MODULES_PATH=../source/opencv_contrib/modules \
-D BUILD_EXAMPLES=ON \
-D INSTALL_C_EXAMPLES=ON \
-D INSTALL_PYTHON_EXAMPLES=ON \
-D OPENCV_GENERATE_PKGCONFIG=ON \
-D OPENCV_ENABLE_NONFREE=ON \
-G "Unix Makefiles" \
-D WITH_CUDA=ON \
-D WITH_CUDNN=ON \
-D OPENCV_DNN_CUDA=ON \
-D ENABLE_FAST_MATH=1 \
-D CUDA_FAST_MATH=1 \
-D WITH_CUBLAS=1 \
-D CUDA_ARCH_BIN=6.1 \
-D HAVE_OPENCV_PYTHON3=ON \
-D PYTHON3_EXECUTABLE=$(which python3) \
-D PYTHON3_INCLUDE_DIR=$(python3 -c "from sysconfig import get_paths; print(get_paths()['include'])") \
-D PYTHON3_NUMPY_INCLUDE_DIRS=$(python3 -c "import numpy; print(numpy.get_include())") \
../source/opencv



//cmake输出中包含 以下内容则可以使用CUDA
NVIDIA CUDA:                   YES (ver 11.4, CUFFT CUBLAS FAST_MATH)
--     NVIDIA GPU arch:             35 37 50 52 60 61 70 75 80 86
--     NVIDIA PTX archs:
--
--   cuDNN:                        YES (ver 8.2.1)
```

Build Directory:

- .. - Parent directory containing the source code (CMakeLists.txt)
- -G "Unix Makefiles" - generator for this project

Build & installation configuration

- -D CMAKE_BUILD_TYPE=RELEASE - Builds an optimized release version (vs debug)
- -D CMAKE_INSTALL_PREFIX=/usr/local - Installation directory (standard Linux location)
- -D OPENCV_EXTRA_MODULES_PATH=../../opencv_contrib/modules - Path to OpenCV's extra/contrib modules (non-free/experimental features)
- -D BUILD_opencv_world=OFF - It combines all OpenCV libraries (core, imgproc, dnn, cudaarithm, etc.) into a single dynamic/static library (e.g., libopencv_world.so). The advantage is easier deployment; the disadvantages are longer compile times and the need to recompile the entire library for any minor change.

Generate Example

- -D BUILD_EXAMPLES=ON - Compiles OpenCV example programs
- -D INSTALL_C_EXAMPLES=ON
  - Copies C and C++ example source code and binaries to the installation directory. Valuable for learning and quick testing of OpenCV features.
- -D INSTALL_PYTHON_EXAMPLES=ON

Developer convenience

- -D OPENCV_GENERATE_PKGCONFIG=ON
  - Essential for Linux developers. Allows easy compilation of OpenCV programs with pkg-config --cflags --libs opencv4. Makes integration with build systems like Makefiles and autotools much simpler.
  - Without this: You need to manually specify include paths (-I/usr/local/include/opencv4) and library paths (-L/usr/local/lib -lopencv_core -lopencv_imgproc ...).
  - With this: Simple compilation becomes: `g++ myprogram.cpp -o myprogram` pkg-config --cflags --libs opencv4``
- -D ENABLE_CXX11=1
  - Ensures modern C++11 features are enabled. Important for compatibility with newer compilers and certain C++11-dependent libraries. Usually auto-detected, but explicit setting prevents issues.

Algorithm availability

- -D OPENCV_ENABLE_NONFREE=True
  - Required for using SIFT, SURF, and other patented algorithms. Without this, these features are disabled due to licensing restrictions. Set to True if you need these algorithms for research/internal use (understand licensing implications).

Python configuration

- -D HAVE_opencv_python3=ON \
- -D PYTHON_EXECUTABLE=~/.virtualenvs/opencv_cuda/bin/python \
- -D PYTHON3_EXECUTABLE=$(which python3) \
- -D PYTHON3_INCLUDE_DIR=$(python3 -c "from sysconfig import get_paths; print(get_paths()['include'])") \
- -D PYTHON3_NUMPY_INCLUDE_DIRS=$(python3 -c "import numpy; print(numpy.get_include())")

CUDA Configuration:

Essential Options:

- -D WITH_CUDA=ON
  - Enables CUDA GPU acceleration
- -D WITH_CUDNN=ON \
  - enable cuDNN (CUDA Deep Neural Network library) support in OpenCV.
- -D OPENCV_DNN_CUDA=ON
  - Strongly recommended to enable. This enables the CUDA backend for OpenCV's deep neural network module (dnn). When enabled, neural network models (like YOLO) loaded with the dnn::Net class will leverage the GPU for inference, resulting in significant speedups.

Performance Optimization:

- -D CUDA_ARCH_BIN=your_compute_capability.
  - This option specifies which GPU architectures to generate Cubin binary code for. The best practice is to specify only the compute capability of your current GPU, for example, 7.5 for your T1200. This maximizes performance for your specific card while avoiding excessively long compile times and oversized binaries.
- -D ENABLE_FAST_MATH=1
  - Enables fast math operations (speed > precision)
- -D CUDA_FAST_MATH=1
  - Enables CUDA-specific fast math optimizations
- -D WITH_CUBLAS=1
  - Enables cuBLAS library for BLAS operations on GPU

Optional:

-
  - -D BUILD_CUDA_STUBS=ON
    - Primarily used for legacy OpenCV versions or special cross-compilation scenarios (e.g., packaging on a Linux system without a GPU). For standard desktop environments compiling CUDA-accelerated OpenCV, it should usually be set to OFF or omitted (default is OFF).
- **CUDA_ARCH_BIN**: Enter the compute capability coefficient of your current computer's graphics card here.
  - You can find the compute capability for your corresponding graphics card here: NVIDIA GPU Compute Capability Table https://developer.nvidia.com/cuda/gpus
  - Visit the link, scroll down to "CUDA-Enabled GeForce and TITAN Products," click on it, and look up your graphics card's compute capability. You will need to fill in the CUDA_ARCH_BIN parameter with this value in the next step.

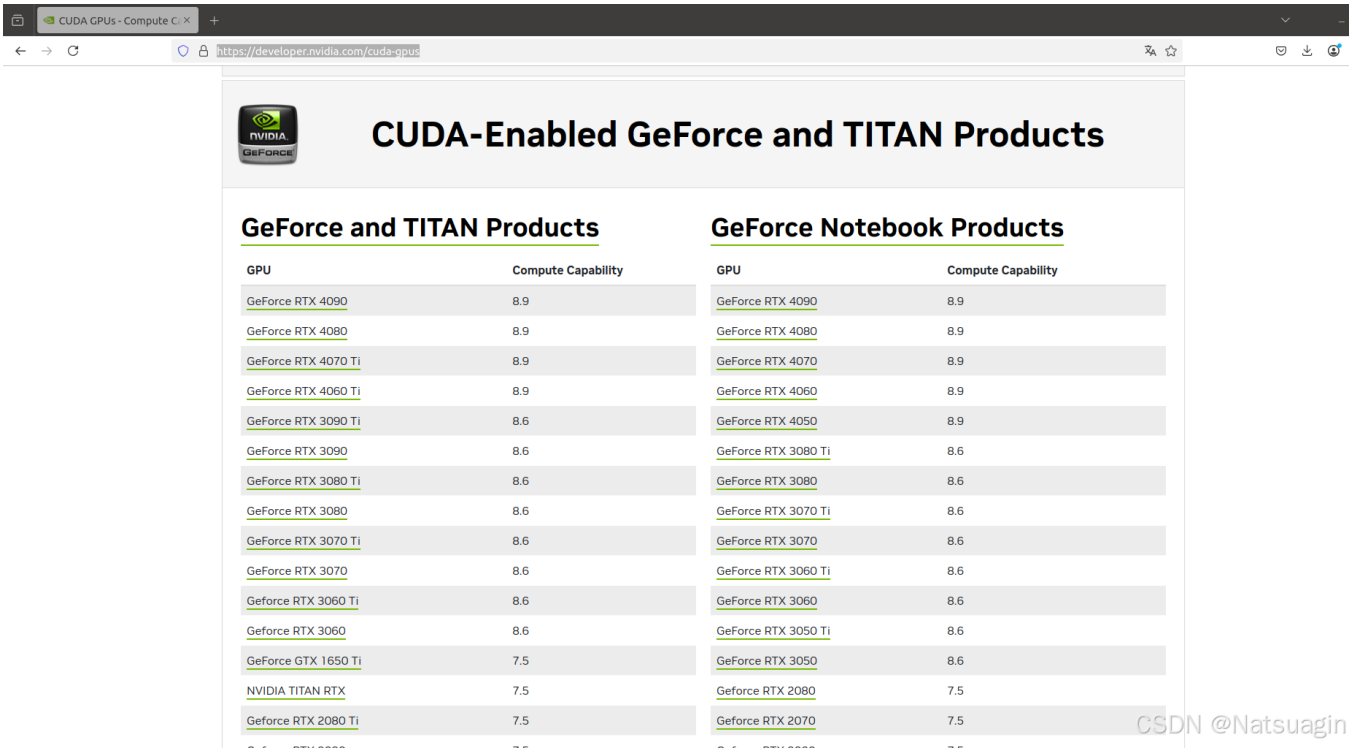
- Use `nvidia-smi -L` to check your NVIDIA GPU model. Go to https://developer.nvidia.com/cuda-gpus and search for the compute capability corresponding to your GPU model. This is the CUDA_ARCH_BIN value. For example, my computer has a T1200, so CUDA_ARCH_BIN = 7.5.
  - GPU 0: NVIDIA TITAN Xp (UUID: GPU-4c49dbc7-5541-25eb-3ab1-e963ff6602d5) corresponds to 6.1

NVIDIA Quadro and NVIDIA RTX Desktop GPUs

GPU	Compute Capability
RTX 6000	8.9
RTX A6000	8.6
RTX A5000	8.6
RTX A4000	8.6
T1000	7.5
T600	7.5
T400	7.5
Quadro RTX 8000	7.5
Quadro RTX 6000	7.5
Quadro RTX 5000	7.5
Quadro RTX 4000	7.5
Quadro GV100	7.0
Quadro GP100	6.0
Quadro P6000	6.1
Quadro P5000	6.1
Quadro P4000	6.1
Quadro P2200	6.1

NVIDIA Quadro and NVIDIA RTX Mobile GPUs

GPU	Compute Capability
RTX A5000	8.6
RTX A4000	8.6
RTX A3000	8.6
RTX A2000	8.6
RTX 5000	7.5
RTX 4000	7.5
RTX 3000	7.5
T2000	7.5
T1200	7.5
T1000	7.5
T600	7.5
T500	7.5
P620	6.1
P520	6.1
Quadro P5200	6.1
Quadro P4200	6.1
Quadro P3200	6.1



---

after run cmake command , we get

1 in the macOS

```
-- SYCL/OpenCL samples are skipped: SYCL SDK is required
--    - check configuration of SYCL_DIR/SYCL_ROOT/CMAKE_MODULE_PATH
--    - ensure that right compiler is selected from SYCL SDK (e.g, clang++): CMAKE_CXX_COMPILER=/usr/bin/c++
CMake Warning (dev) at samples/CMakeLists.txt:13 (install):
  Policy CMP0177 is not set: install() DESTINATION paths are normalized.  Run
  "cmake --help-policy CMP0177" for policy details.  Use the cmake_policy
  command to set the policy and suppress this warning.
Call Stack (most recent call first):
  samples/CMakeLists.txt:54 (ocv_install_example_src)
This warning is for project developers.  Use -Wno-dev to suppress it.

--
-- General configuration for OpenCV 4.13.0 =====================================
--   Version control:               4.13.0
--
--   Extra modules:
--     Location (extra):            /Users/yanbo/Documents/Code_Storage/opencv/source/opencv_contrib/modules
--     Version control (extra):     4.13.0
--
--   Platform:
--     Timestamp:                   2026-02-11T19:21:04Z
--     Host:                        Darwin 22.6.0 x86_64
--     CMake:                       4.2.3
--     CMake generator:             Unix Makefiles
--     CMake build tool:            /usr/bin/make
--     Configuration:               RELEASE
--     Algorithm Hint:              ALGO_HINT_ACCURATE
--
--   CPU/HW features:
--     Baseline:                    SSE SSE2 SSE3 SSSE3 SSE4_1
--       requested:                 DETECT
--     Dispatched code generation:  SSE4_2 AVX FP16 AVX2 AVX512_SKX
--       requested:                 SSE4_1 SSE4_2 AVX FP16 AVX2 AVX512_SKX
--       SSE4_2 (2 files):          + POPCNT SSE4_2
--       AVX (10 files):            + POPCNT SSE4_2 AVX
--       FP16 (1 files):            + POPCNT SSE4_2 AVX FP16
--       AVX2 (39 files):           + POPCNT SSE4_2 AVX FP16 AVX2 FMA3
--       AVX512_SKX (10 files):     + POPCNT SSE4_2 AVX FP16 AVX2 FMA3 AVX_512F AVX512_COMMON AVX512_SKX
--
--   C/C++:
--     Built as dynamic libs?:      YES
--     C++ standard:                11
--     C++ Compiler:                /usr/bin/c++  (ver 14.0.0.14000029)
--     C++ flags (Release):         -fsigned-char -W -Wall -Wreturn-type -Wnon-virtual-dtor -Waddress -
Wsequence-point -Wformat -Wformat-security -Wmissing-declarations -Wmissing-prototypes -Wstrict-prototypes -
Wundef -Winit-self -Wpointer-arith -Wshadow -Wsign-promo -Wuninitialized -Wno-delete-non-virtual-dtor -Wno-
unnamed-type-template-args -Wno-comment -fdiagnostics-show-option -Qunused-arguments -Wno-semicolon-before-
method-body -ffunction-sections -fdata-sections  -fvisibility=hidden -fvisibility-inlines-hidden -Wno-
deprecated-copy -O3 -DNDEBUG  -DNDEBUG
--     C++ flags (Debug):           -fsigned-char -W -Wall -Wreturn-type -Wnon-virtual-dtor -Waddress -
Wsequence-point -Wformat -Wformat-security -Wmissing-declarations -Wmissing-prototypes -Wstrict-prototypes -
Wundef -Winit-self -Wpointer-arith -Wshadow -Wsign-promo -Wuninitialized -Wno-delete-non-virtual-dtor -Wno-
unnamed-type-template-args -Wno-comment -fdiagnostics-show-option -Qunused-arguments -Wno-semicolon-before-
method-body -ffunction-sections -fdata-sections  -fvisibility=hidden -fvisibility-inlines-hidden -Wno-
deprecated-copy -g  -O0 -DDEBUG -D_DEBUG
--     C Compiler:                  /usr/bin/cc
--     C flags (Release):           -fsigned-char -W -Wall -Wreturn-type -Wnon-virtual-dtor -Waddress -
Wsequence-point -Wformat -Wformat-security -Wmissing-declarations -Wmissing-prototypes -Wstrict-prototypes -
Wundef -Winit-self -Wpointer-arith -Wshadow -Wsign-promo -Wuninitialized -Wno-delete-non-virtual-dtor -Wno-
unnamed-type-template-args -Wno-comment -fdiagnostics-show-option -Qunused-arguments -Wno-semicolon-before-
method-body -ffunction-sections -fdata-sections  -fvisibility=hidden -fvisibility-inlines-hidden -O3 -DNDEBUG
-DNDEBUG
--     C flags (Debug):             -fsigned-char -W -Wall -Wreturn-type -Wnon-virtual-dtor -Waddress -
Wsequence-point -Wformat -Wformat-security -Wmissing-declarations -Wmissing-prototypes -Wstrict-prototypes -
Wundef -Winit-self -Wpointer-arith -Wshadow -Wsign-promo -Wuninitialized -Wno-delete-non-virtual-dtor -Wno-
unnamed-type-template-args -Wno-comment -fdiagnostics-show-option -Qunused-arguments -Wno-semicolon-before-
method-body -ffunction-sections -fdata-sections  -fvisibility=hidden -fvisibility-inlines-hidden -g  -O0 -
DDEBUG -D_DEBUG
--     Linker flags (Release):      -L/usr/local/opt/ruby/lib  -Wl,-dead_strip
--     Linker flags (Debug):        -L/usr/local/opt/ruby/lib  -Wl,-dead_strip
```

```
--     ccache:                      NO
--     Precompiled headers:         NO
--     Extra dependencies:
--     3rdparty dependencies:
--
--   OpenCV modules:
--     To be built:                 alphamat aruco bgsegm bioinspired calib3d ccalib core datasets dnn
dnn_objdetect dnn_superres dpm face features2d flann freetype fuzzy gapi hfs highgui img_hash imgcodecs
imgproc intensity_transform java line_descriptor mcc ml objdetect optflow phase_unwrapping photo plot quality
rapid reg rgbd saliency shape signal stereo stitching structured_light superres surface_matching text
tracking ts video videoio videostab wechat_qrcode xfeatures2d ximgproc xobjdetect xphoto
--     Disabled:                    world
--     Disabled by dependency:      -
--     Unavailable:                 cannops cudaarithm cudabgsegm cudacodec cudafeatures2d cudafilters
cudaimgproc cudalegacy cudaobjdetect cudaoptflow cudastereo cudawarping cudev cvv fastcv hdf julia matlab
ovis python2 python3 sfm viz
--     Applications:                tests perf_tests examples apps
--     Documentation:               NO
--     Non-free algorithms:         YES
--
--   GUI:                           COCOA
--     Cocoa:                       YES
--     VTK support:                 NO
--
--   Media I/O:
--     ZLib:                        build (ver 1.3.1)
--     JPEG:                        build-libjpeg-turbo (ver 3.1.2-70)
--       SIMD Support Request:      YES
--       SIMD Support:              YES
--     WEBP:                        build (ver decoder: 0x0210, encoder: 0x0210, demux: 0x0107)
--     AVIF:                        avif (ver 0.11.1)
--     PNG:                         build (ver 1.6.53)
--       SIMD Support Request:      YES
--       SIMD Support:              YES (Intel SSE)
--     TIFF:                        build (ver 42 - 4.7.1)
--     JPEG 2000:                   build (ver 2.5.3)
--     OpenEXR:                     OpenEXR::OpenEXR (ver 3.4.4)
--     GIF:                         YES
--     HDR:                         YES
--     SUNRASTER:                   YES
--     PXM:                         YES
--     PFM:                         YES
--
--   Video I/O:
--     FFMPEG:                      YES
--       avcodec:                   YES (62.11.100)
--       avformat:                  YES (62.3.100)
--       avutil:                    YES (60.8.100)
--       swscale:                   YES (9.1.100)
--       avdevice:                  YES (62.1.100)
--     GStreamer:                   NO
--     AVFoundation:                YES
--     Orbbec:                      NO
--
--   Parallel framework:            GCD
--
--   Trace:                         YES (with Intel ITT(3.25.4))
--
--   Other third-party libraries:
--     Intel IPP:                   2021.9.1 [2021.9.1]
--            at:
/Users/yanbo/Documents/Code_Storage/opencv/source/opencv/build/3rdparty/ippicv/ippicv_mac/icv
--     Intel IPP IW:                sources (2021.9.1)
--               at:
/Users/yanbo/Documents/Code_Storage/opencv/source/opencv/build/3rdparty/ippicv/ippicv_mac/iw
--     Lapack:                      YES (-framework Accelerate)
--     Eigen:                       YES (ver 5.0.1)
--     Custom HAL:                  YES (ipp (ver 0.0.1))
--     Protobuf:                    build (3.19.1)
--     Flatbuffers:                 builtin/3rdparty (25.9.23)
--
```

```
--   OpenCL:                        YES (no extra features)
--     Include path:                NO
--     Link libraries:              -framework OpenCL
--
--   Python (for build):            /usr/local/bin/python3
--
--   Java:
--     ant:                         NO
--     Java:                        YES (ver 11.0.17)
--     JNI:                         /Library/Java/JavaVirtualMachines/jdk-11.0.17.jdk/Contents/Home/include
/Library/Java/JavaVirtualMachines/jdk-11.0.17.jdk/Contents/Home/include/darwin
/Library/Java/JavaVirtualMachines/jdk-11.0.17.jdk/Contents/Home/include
--     Java wrappers:               YES (JAVA)
--     Java tests:                  NO
--
--   Install to:                    /usr/local
-- -----------------------------------------------------------------
--
-- Configuring done (172.7s)
-- Generating done (10.1s)
-- Build files have been written to: /Users/yanbo/Documents/Code_Storage/opencv/source/opencv/build
 yanbo@Yanbos-MacBook-Pro ▶ ~/Documents/Code_Storage/opencv/source/opencv/build ▶ ➦ fe38fc608f ▶ cmake .. \
-D CMAKE_BUILD_TYPE=RELEASE \
-D CMAKE_INSTALL_PREFIX=/usr/local \
-D OPENCV_EXTRA_MODULES_PATH=../../opencv_contrib/modules \
-D BUILD_EXAMPLES=ON \
-D INSTALL_C_EXAMPLES=ON \
-D INSTALL_PYTHON_EXAMPLES=ON \
-D OPENCV_GENERATE_PKGCONFIG=ON \
-D OPENCV_ENABLE_NONFREE=ON
CMake Deprecation Warning at CMakeLists.txt:25 (cmake_minimum_required):
  Compatibility with CMake < 3.10 will be removed from a future version of
  CMake.

  Update the VERSION argument <min> value.  Or, use the <min>...<max> syntax
  to tell CMake that the project requires at least <min> but has been updated
  to work with policies introduced by <max> or earlier.


-- Detected processor: x86_64
-- Looking for ccache - not found
-- AVX512_KNM is not supported by C++ compiler
-- libjpeg-turbo: VERSION = 3.1.2, BUILD = opencv-4.13.0-libjpeg-turbo
-- Performing Test HAVE_BUILTIN_CTZL
-- Performing Test HAVE_BUILTIN_CTZL - Success
-- CMAKE_ASM_NASM_COMPILER = /usr/local/bin/nasm
-- CMAKE_ASM_NASM_OBJECT_FORMAT = macho64
-- CMAKE_ASM_NASM_FLAGS =  -DMACHO -D__x86_64__ -DPIC
-- SIMD extensions: x86_64 (WITH_SIMD = 1)
-- Could NOT find OpenJPEG (minimal suitable version: 2.0, recommended version >= 2.3.1). OpenJPEG will be
built from sources
-- OpenJPEG: VERSION = 2.5.3, BUILD = opencv-4.13.0-openjp2-2.5.3
-- OpenJPEG libraries will be built from sources: libopenjp2 (version "2.5.3")
-- found Intel IPP (ICV version): 2021.9.1 [2021.9.1]
-- at: /Users/yanbo/Documents/Code_Storage/opencv/source/opencv/build/3rdparty/ippicv/ippicv_mac/icv
-- found Intel IPP Integration Wrappers sources: 2021.9.1
-- at: /Users/yanbo/Documents/Code_Storage/opencv/source/opencv/build/3rdparty/ippicv/ippicv_mac/iw
-- LAPACK(Unknown): LAPACK_LIBRARIES: -framework Accelerate
-- Looking for Accelerate/Accelerate.h
-- Looking for Accelerate/Accelerate.h - found
-- Looking for Accelerate/Accelerate.h
-- Looking for Accelerate/Accelerate.h - found
-- LAPACK(Unknown): Support is enabled.
-- Could NOT find Pylint (missing: PYLINT_EXECUTABLE)
-- Could NOT find Flake8 (missing: FLAKE8_EXECUTABLE)
-- VTK is not found. Please set -DVTK_DIR in CMake to VTK build directory, or to VTK install subdirectory
with VTKConfig.cmake file
-- Checking for module 'gstreamer-base-1.0'
--   Package 'gstreamer-base-1.0' not found
-- Checking for module 'gstreamer-app-1.0'
--   Package 'gstreamer-app-1.0' not found
```

```
-- Checking for module 'gstreamer-riff-1.0'
--   Package 'gstreamer-riff-1.0' not found
-- Checking for module 'gstreamer-pbutils-1.0'
--   Package 'gstreamer-pbutils-1.0' not found
-- Checking for module 'gstreamer-video-1.0'
--   Package 'gstreamer-video-1.0' not found
-- Checking for module 'gstreamer-audio-1.0'
--   Package 'gstreamer-audio-1.0' not found
-- freetype2:   YES (ver 24.3.18)
-- harfbuzz:    YES (ver 5.3.1)
-- Could NOT find HDF5 (missing: HDF5_LIBRARIES HDF5_INCLUDE_DIRS)
-- Julia not found. Not compiling Julia Bindings.
-- Module opencv_ovis disabled because OGRE3D was not found
-- No preference for use of exported gflags CMake configuration set, and no hints for include/library
directories provided. Defaulting to preferring an installed/exported gflags CMake configuration if available.
-- Failed to find installed gflags CMake configuration, searching for gflags build directories exported with
CMake.
-- Failed to find gflags - Failed to find an installed/exported CMake configuration for gflags, will perform
search for installed gflags components.
-- Failed to find gflags - Could not find gflags include directory, set GFLAGS_INCLUDE_DIR to directory
containing gflags/gflags.h
-- Failed to find glog - Could not find glog include directory, set GLOG_INCLUDE_DIR to directory containing
glog/logging.h
-- Module opencv_sfm disabled because the following dependencies are not found: Glog/Gflags
-- Tesseract:   YES (ver 5.2.0)
-- Allocator metrics storage type: 'long long'
-- Excluding from source files list: <BUILD>/modules/core/test/test_intrin256.lasx.cpp
-- Excluding from source files list: modules/imgproc/src/imgwarp.lasx.cpp
-- Excluding from source files list: modules/imgproc/src/resize.lasx.cpp
-- Registering hook 'INIT_MODULE_SOURCES_opencv_dnn':
/Users/yanbo/Documents/Code_Storage/opencv/source/opencv/modules/dnn/cmake/hooks/INIT_MODULE_SOURCES_opencv_d
nn.cmake
-- opencv_dnn: filter out ocl4dnn source code
-- opencv_dnn: filter out cuda4dnn source code
-- Excluding from source files list: <BUILD>/modules/dnn/layers/layers_common.rvv.cpp
-- Excluding from source files list: <BUILD>/modules/dnn/layers/layers_common.lasx.cpp
-- Excluding from source files list: <BUILD>/modules/dnn/layers/layers_common.neon.cpp
-- Excluding from source files list: <BUILD>/modules/dnn/layers/layers_common.sve.cpp
-- Excluding from source files list: <BUILD>/modules/dnn/int8layers/layers_common.rvv.cpp
-- Excluding from source files list: <BUILD>/modules/dnn/int8layers/layers_common.lasx.cpp
-- Excluding from source files list: <BUILD>/modules/dnn/int8layers/layers_common.neon.cpp
-- Excluding from source files list: <BUILD>/modules/dnn/layers/cpu_kernels/conv_block.neon.cpp
-- Excluding from source files list: <BUILD>/modules/dnn/layers/cpu_kernels/conv_block.neon_fp16.cpp
-- Excluding from source files list: <BUILD>/modules/dnn/layers/cpu_kernels/conv_depthwise.rvv.cpp
-- Excluding from source files list: <BUILD>/modules/dnn/layers/cpu_kernels/conv_depthwise.lasx.cpp
-- Excluding from source files list: <BUILD>/modules/dnn/layers/cpu_kernels/fast_gemm_kernels.neon.cpp
-- Excluding from source files list: <BUILD>/modules/dnn/layers/cpu_kernels/fast_gemm_kernels.lasx.cpp
-- highgui: using builtin backend: COCOA
-- Use autogenerated whitelist
/Users/yanbo/Documents/Code_Storage/opencv/source/opencv/build/modules/js_bindings_generator/whitelist.json
-- Set Cleaners to True
-- Found 'misc' Python modules from
/Users/yanbo/Documents/Code_Storage/opencv/source/opencv/modules/python/package/extra_modules
-- Found 'mat_wrapper;utils' Python modules from
/Users/yanbo/Documents/Code_Storage/opencv/source/opencv/modules/core/misc/python/package
-- Found 'gapi' Python modules from
/Users/yanbo/Documents/Code_Storage/opencv/source/opencv/modules/gapi/misc/python/package
CMake Deprecation Warning at samples/sycl/CMakeLists.txt:24 (cmake_policy):
  Compatibility with CMake < 3.10 will be removed from a future version of
  CMake.

  Update the VERSION argument <min> value.  Or, use the <min>...<max> syntax
  to tell CMake that the project requires at least <min> but has been updated
  to work with policies introduced by <max> or earlier.


-- SYCL/OpenCL samples are skipped: SYCL SDK is required
--     - check configuration of SYCL_DIR/SYCL_ROOT/CMAKE_MODULE_PATH
--     - ensure that right compiler is selected from SYCL SDK (e.g, clang++): CMAKE_CXX_COMPILER=/usr/bin/c++
CMake Warning (dev) at samples/CMakeLists.txt:13 (install):
  Policy CMP0177 is not set: install() DESTINATION paths are normalized.  Run
```

```
  "cmake --help-policy CMP0177" for policy details.  Use the cmake_policy
  command to set the policy and suppress this warning.
Call Stack (most recent call first):
  samples/CMakeLists.txt:54 (ocv_install_example_src)
This warning is for project developers.  Use -Wno-dev to suppress it.

--
-- General configuration for OpenCV 4.13.0 =====================================
--   Version control:               4.13.0
--
--   Extra modules:
--     Location (extra):            /Users/yanbo/Documents/Code_Storage/opencv/source/opencv_contrib/modules
--     Version control (extra):     4.13.0
--
--   Platform:
--     Timestamp:                   2026-02-11T19:21:04Z
--     Host:                        Darwin 22.6.0 x86_64
--     CMake:                       4.2.3
--     CMake generator:             Unix Makefiles
--     CMake build tool:            /usr/bin/make
--     Configuration:               RELEASE
--     Algorithm Hint:              ALGO_HINT_ACCURATE
--
--   CPU/HW features:
--     Baseline:                    SSE SSE2 SSE3 SSSE3 SSE4_1
--       requested:                 DETECT
--     Dispatched code generation:  SSE4_2 AVX FP16 AVX2 AVX512_SKX
--       requested:                 SSE4_1 SSE4_2 AVX FP16 AVX2 AVX512_SKX
--       SSE4_2 (2 files):          + POPCNT SSE4_2
--       AVX (10 files):            + POPCNT SSE4_2 AVX
--       FP16 (1 files):            + POPCNT SSE4_2 AVX FP16
--       AVX2 (39 files):           + POPCNT SSE4_2 AVX FP16 AVX2 FMA3
--       AVX512_SKX (10 files):     + POPCNT SSE4_2 AVX FP16 AVX2 FMA3 AVX_512F AVX512_COMMON AVX512_SKX
--
--   C/C++:
--     Built as dynamic libs?:      YES
--     C++ standard:                11
--     C++ Compiler:                /usr/bin/c++  (ver 14.0.0.14000029)
--     C++ flags (Release):         -fsigned-char -W -Wall -Wreturn-type -Wnon-virtual-dtor -Waddress -
Wsequence-point -Wformat -Wformat-security -Wmissing-declarations -Wmissing-prototypes -Wstrict-prototypes -
Wundef -Winit-self -Wpointer-arith -Wshadow -Wsign-promo -Wuninitialized -Wno-delete-non-virtual-dtor -Wno-
unnamed-type-template-args -Wno-comment -fdiagnostics-show-option -Qunused-arguments -Wno-semicolon-before-
method-body -ffunction-sections -fdata-sections  -fvisibility=hidden -fvisibility-inlines-hidden -Wno-
deprecated-copy -O3 -DNDEBUG  -DNDEBUG
--     C++ flags (Debug):           -fsigned-char -W -Wall -Wreturn-type -Wnon-virtual-dtor -Waddress -
Wsequence-point -Wformat -Wformat-security -Wmissing-declarations -Wmissing-prototypes -Wstrict-prototypes -
Wundef -Winit-self -Wpointer-arith -Wshadow -Wsign-promo -Wuninitialized -Wno-delete-non-virtual-dtor -Wno-
unnamed-type-template-args -Wno-comment -fdiagnostics-show-option -Qunused-arguments -Wno-semicolon-before-
method-body -ffunction-sections -fdata-sections  -fvisibility=hidden -fvisibility-inlines-hidden -Wno-
deprecated-copy -g  -O0 -DDEBUG -D_DEBUG
--     C Compiler:                  /usr/bin/cc
--     C flags (Release):           -fsigned-char -W -Wall -Wreturn-type -Wnon-virtual-dtor -Waddress -
Wsequence-point -Wformat -Wformat-security -Wmissing-declarations -Wmissing-prototypes -Wstrict-prototypes -
Wundef -Winit-self -Wpointer-arith -Wshadow -Wsign-promo -Wuninitialized -Wno-delete-non-virtual-dtor -Wno-
unnamed-type-template-args -Wno-comment -fdiagnostics-show-option -Qunused-arguments -Wno-semicolon-before-
method-body -ffunction-sections -fdata-sections  -fvisibility=hidden -fvisibility-inlines-hidden -O3 -DNDEBUG
-DNDEBUG
--     C flags (Debug):             -fsigned-char -W -Wall -Wreturn-type -Wnon-virtual-dtor -Waddress -
Wsequence-point -Wformat -Wformat-security -Wmissing-declarations -Wmissing-prototypes -Wstrict-prototypes -
Wundef -Winit-self -Wpointer-arith -Wshadow -Wsign-promo -Wuninitialized -Wno-delete-non-virtual-dtor -Wno-
unnamed-type-template-args -Wno-comment -fdiagnostics-show-option -Qunused-arguments -Wno-semicolon-before-
method-body -ffunction-sections -fdata-sections  -fvisibility=hidden -fvisibility-inlines-hidden -g  -O0 -
DDEBUG -D_DEBUG
--     Linker flags (Release):      -L/usr/local/opt/ruby/lib  -Wl,-dead_strip
--     Linker flags (Debug):        -L/usr/local/opt/ruby/lib  -Wl,-dead_strip
--     ccache:                      NO
--     Precompiled headers:         NO
--     Extra dependencies:
--     3rdparty dependencies:
--
--   OpenCV modules:
```

```
--     To be built:                 alphamat aruco bgsegm bioinspired calib3d ccalib core datasets dnn
dnn_objdetect dnn_superres dpm face features2d flann freetype fuzzy gapi hfs highgui img_hash imgcodecs
imgproc intensity_transform java line_descriptor mcc ml objdetect optflow phase_unwrapping photo plot quality
rapid reg rgbd saliency shape signal stereo stitching structured_light superres surface_matching text
tracking ts video videoio videostab wechat_qrcode xfeatures2d ximgproc xobjdetect xphoto
--     Disabled:                    world
--     Disabled by dependency:      -
--     Unavailable:                 cannops cudaarithm cudabgsegm cudacodec cudafeatures2d cudafilters
cudaimgproc cudalegacy cudaobjdetect cudaoptflow cudastereo cudawarping cudev cvv fastcv hdf julia matlab
ovis python2 python3 sfm viz
--     Applications:                tests perf_tests examples apps
--     Documentation:               NO
--     Non-free algorithms:         YES
--
--   GUI:                           COCOA
--     Cocoa:                       YES
--     VTK support:                 NO
--
--   Media I/O:
--     ZLib:                        build (ver 1.3.1)
--     JPEG:                        build-libjpeg-turbo (ver 3.1.2-70)
--       SIMD Support Request:      YES
--       SIMD Support:              YES
--     WEBP:                        build (ver decoder: 0x0210, encoder: 0x0210, demux: 0x0107)
--     AVIF:                        avif (ver 0.11.1)
--     PNG:                         build (ver 1.6.53)
--       SIMD Support Request:      YES
--       SIMD Support:              YES (Intel SSE)
--     TIFF:                        build (ver 42 - 4.7.1)
--     JPEG 2000:                   build (ver 2.5.3)
--     OpenEXR:                     OpenEXR::OpenEXR (ver 3.4.4)
--     GIF:                         YES
--     HDR:                         YES
--     SUNRASTER:                   YES
--     PXM:                         YES
--     PFM:                         YES
--
--   Video I/O:
--     FFMPEG:                      YES
--       avcodec:                   YES (62.11.100)
--       avformat:                  YES (62.3.100)
--       avutil:                    YES (60.8.100)
--       swscale:                   YES (9.1.100)
--       avdevice:                  YES (62.1.100)
--     GStreamer:                   NO
--     AVFoundation:                YES
--     Orbbec:                      NO
--
--   Parallel framework:            GCD
--
--   Trace:                         YES (with Intel ITT(3.25.4))
--
--   Other third-party libraries:
--     Intel IPP:                   2021.9.1 [2021.9.1]
--            at:
/Users/yanbo/Documents/Code_Storage/opencv/source/opencv/build/3rdparty/ippicv/ippicv_mac/icv
--     Intel IPP IW:                sources (2021.9.1)
--               at:
/Users/yanbo/Documents/Code_Storage/opencv/source/opencv/build/3rdparty/ippicv/ippicv_mac/iw
--     Lapack:                      YES (-framework Accelerate)
--     Eigen:                       YES (ver 5.0.1)
--     Custom HAL:                  YES (ipp (ver 0.0.1))
--     Protobuf:                    build (3.19.1)
--     Flatbuffers:                 builtin/3rdparty (25.9.23)
--
--   OpenCL:                        YES (no extra features)
--     Include path:                NO
--     Link libraries:              -framework OpenCL
--
--   Python (for build):            /usr/local/bin/python3
--
```

```
--   Java:
--     ant:                         NO
--     Java:                        YES (ver 11.0.17)
--     JNI:                         /Library/Java/JavaVirtualMachines/jdk-11.0.17.jdk/Contents/Home/include
/Library/Java/JavaVirtualMachines/jdk-11.0.17.jdk/Contents/Home/include/darwin
/Library/Java/JavaVirtualMachines/jdk-11.0.17.jdk/Contents/Home/include
--     Java wrappers:               YES (JAVA)
--     Java tests:                  NO
--
--   Install to:                    /usr/local
-- -----------------------------------------------------------------
--
-- Configuring done (16.0s)
-- Generating done (8.6s)
-- Build files have been written to: /Users/yanbo/Documents/Code_Storage/opencv/source/opencv/build
```

---

2 in the linux

```
CMake Warning at samples/samples_utils.cmake:10 (add_executable):
  Cannot generate a safe runtime search path for target
  example_opencl_opencl-opencv-interop because files in some directories may
  conflict with libraries in implicit directories:

    runtime library [libOpenCL.so.1] in /usr/lib/x86_64-linux-gnu may be hidden by files in:
      /usr/local/cuda/lib64

  Some of these libraries may not be found correctly.
Call Stack (most recent call first):
  samples/opencl/CMakeLists.txt:33 (ocv_define_sample)
```

1.2.2 after cmake, make it again

```
cd build

# make -j16 indicate excute 16 jobs simultanously
# Compilation only - files are in build/ directory
#  in maxos
make -j$(sysctl -n hw.ncpu)

# in linux
make -j$(nproc)

#  MUST RUN THIS - copies files to /usr/local/
make install
```

all complied files is saved in build/install

---

after runs cmake

```
build/
├── lib/                    # Temporary library files generated during compilation (.dylib)
├── bin/                    # Temporary executable files generated during compilation
├── modules/                # Intermediate compilation files for each module
│   ├── core/              # .o files for the core module
│   ├── imgproc/           # .o files for the imgproc module
│   └── ...
├── libopencv_core.4.13.0.dylib  # Library files generated from compilation
├── opencv_version         # Version test program
└── CMakeCache.txt         # CMake cache configuration
```

After running sudo make install, files are copied HERE /usr/local/ (set by -D CMAKE_INSTALL_PREFIX=/usr/local)

```
/usr/local/
├── lib/                    # ✅ FINAL LOCATION - Libraries
│   ├── libopencv_core.4.13.0.dylib
│   ├── libopencv_imgproc.4.13.0.dylib
│   ├── libopencv_dnn.4.13.0.dylib
│   ├── libopencv_contrib*.dylib  # Contrib modules
│   └── pkgconfig/          # pkg-config configuration
│       └── opencv4.pc
│
├── include/                # ✅ FINAL LOCATION - Headers
│   └── opencv4/
│       └── opencv2/
│           ├── opencv.hpp
│           ├── core.hpp
│           └── ...
│
├── bin/                    # Executables
│   ├── opencv_version
│   └── opencv_*_demo
│
└── share/                 # Examples and documentation
    └── opencv4/
        └── examples/
```

1.2.3 Opencv environment variable in Linux

OPENCV_DIR:C:\opencv\build\x64\vc14；

```
# Step 1: Configure the library path
# What it does: Creates/edits a configuration file for the dynamic linker/loader
sudo gedit /etc/ld.so.conf.d/opencv.conf

# File content to add:
/usr/local/lib
# This tells the system to look for shared libraries (.so files) in /usr/local/lib where OpenCV libraries
were installed.


# Updates the shared library cache
# Rebuilds /etc/ld.so.cache
# Makes the system immediately aware of new libraries in /usr/local/lib
# Without this, you'd get error while loading shared libraries when running OpenCV programs
sudo ldconfig

# Check if OpenCV libraries are found by the linker
ldconfig -p | grep opencv



# Step 2: Configure pkg-config path
sudo gedit /etc/bash.bashrc

# File content to add:
PKG_CONFIG_PATH=$PKG_CONFIG_PATH:/usr/local/lib/pkgconfig
export PKG_CONFIG_PATH

# update
source /etc/bash.bashrc

# Check if pkg-config can find OpenCV
pkg-config --modversion opencv4



##### Step3:  Test OpenCV in Python
python3 -c "import cv2; print(cv2.__version__)"
```

1.2.4 Opencv environment variable in MacOS

Step 1: Configure Dynamic Library Path

```
# 1. macOS does not require separate configuration of library paths, as /usr/local/lib is already in the
default search path
# Verification method:
otool -L /usr/local/lib/libopencv_core.dylib   # Check library dependencies

It returns
/usr/local/lib/libopencv_core.dylib:
    @rpath/libopencv_core.413.dylib (compatibility version 413.0.0, current version 4.13.0)
    /System/Library/Frameworks/OpenCL.framework/Versions/A/OpenCL (compatibility version 1.0.0, current
version 1.0.0)
    /System/Library/Frameworks/Accelerate.framework/Versions/A/Accelerate (compatibility version 1.0.0,
current version 4.0.0)
    /usr/lib/libSystem.B.dylib (compatibility version 1.0.0, current version 1319.0.0)
    /usr/lib/libc++.1.dylib (compatibility version 1.0.0, current version 1300.32.0)
```

if it needs to add library paths,

```
# Edit dynamic linker environment variable (temporarily effective)
export DYLD_LIBRARY_PATH=/usr/local/lib:$DYLD_LIBRARY_PATH

# Make it permanent (add to ~/.zshrc)
echo 'export DYLD_LIBRARY_PATH=/usr/local/lib:$DYLD_LIBRARY_PATH' >> ~/.zshrc
source ~/.zshrc
```

Step 2: configure pkg-config path

```
# 1. Edit the zsh configuration file
nano ~/.zshrc
# Or use another editor: vim ~/.zshrc or code ~/.zshrc

# 2. Add the following content:
export PKG_CONFIG_PATH="/usr/local/lib/pkgconfig:$PKG_CONFIG_PATH"

# 3. Apply the configuration
source ~/.zshrc

# 4. Verify
echo $PKG_CONFIG_PATH    # Should contain /usr/local/lib/pkgconfig
```

Step3: verify OpenCV installation

```
# 1. Check if pkg-config can find OpenCV
pkg-config --modversion opencv4

# If it fails, try opencv (without number):
pkg-config --modversion opencv

# 2. Check if library files exist
ls -la /usr/local/lib/libopencv_*    # View installed OpenCV library files

# 3. Check Python bindings (if Python support was compiled)
python3 -c "import cv2; print(cv2.__version__)"

# 4. View library dependencies
otool -L /usr/local/lib/libopencv_core.dylib | grep cuda  # Check CUDA support (if enabled)
```

1.3 Test

1.3.1

In the opencv/samples/gpu directory, execute any .exe program

```
pkg-config opencv --modversion
$4.5.5
```

/usr/local/lib/pkgconfig/ contain opencv.pc

```
prefix=/usr/local
exec_prefix=${prefix}
includedir=/usr/local/include
libdir=/usr/local/lib

Name: OpenCV
Description: Open Source Computer Vision Library
Version: 4.5.5
Libs: -L${exec_prefix}/lib -lopencv_stitching -lopencv_superres -lopencv_videostab -lopencv_aruco -
lopencv_bgsegm -lopencv_bioinspired -lopencv_ccalib -lopencv_dnn_objdetect -lopencv_dpm -lopencv_face -
lopencv_photo -lopencv_freetype -lopencv_fuzzy -lopencv_hdf -lopencv_hfs -lopencv_mcc -lopencv_sfm -
lopencv_img_hash -lopencv_line_descriptor -lopencv_optflow -lopencv_reg -lopencv_rgbd -lopencv_saliency -
lopencv_stereo -lopencv_structured_light -lopencv_phase_unwrapping -lopencv_surface_matching -
lopencv_tracking -lopencv_datasets -lopencv_text -lopencv_dnn -lopencv_plot -lopencv_xfeatures2d -
lopencv_shape -lopencv_video -lopencv_ml -lopencv_ximgproc -lopencv_calib3d -lopencv_features2d -
lopencv_highgui -lopencv_videoio -lopencv_flann -lopencv_xobjdetect -lopencv_imgcodecs -lopencv_objdetect -
lopencv_xphoto -lopencv_imgproc -lopencv_core -lopencv_viz -lopencv_gapi -lopencv_cudev -lopencv_rapid -
lopencv_stereo -lopencv_barcode -lopencv_quality -lopencv_alphamat -lopencv_cudacodec -lopencv_cudaarithm -
lopencv_cudabgsegm -lopencv_cudalegacy -lopencv_cudastereo -lopencv_cudafilters -lopencv_cudaimgproc -
lopencv_cudaoptflow -lopencv_cudawraping -lopencv_dnn_supress -lopencv_cudaobjdetect -lopencv_wechat_qrcode -
lopencv_cudafeatures2d -lopencv_intensity_transform
Libs.private: -ldl -lm -lpthread -lrt
Cflags: -I${includedir}
```

1.3.2 Using OpenCV to Process Images

1. Create a new folder, I named it OpenCV-Test
2. Enter the folder, then right-click to open the terminal
3. Use 'touch main.cpp' to create a file named main.cpp
4. Right-click on main.cpp and open it with an IDE
5. Copy the code:

```
#include <opencv2/opencv.hpp>
#include <iostream>

using namespace cv;
using namespace std;

int main(int argc, char** argv)
{
    // Read image
    Mat image = imread("OpenCV_Logo.png");

    // Check for failure
    if (image.empty())
    {
        cout << "Could not open or find the image" << endl;
        cin.get(); // Wait for keyboard input
        return -1;
    }

    String windowName = "OpenCV Test";     // Window name
    namedWindow(windowName);                // Create new window
    imshow(windowName, image);              // Display image in the new window
    waitKey(0);                             // Wait for keyboard input
    destroyWindow(windowName);               // Close all windows
```

```
    return 0;
}
```

6. Laden Sie ein beliebiges Bild aus dem Internet herunter und legen Sie es in denselben Ordner wie main.cpp.
7. Ändern Sie den Bildpfad.
8. Mit g++ kompilieren.

```
g++ main.cpp `pkg-config --libs --cflags opencv4` -o main

g++ main.cpp 'pkg-config --libs --cflags opencv4'  -o out -std=c++11
```

9. run programm
   ./main

1.3.3 Verify if C++ supports CUDA-enabled OpenCV

```
创建 test_opencv.cpp
sudo nano test_opencv.cpp
```

test_opencv.cpp

```
#include <opencv2/cvconfig.h>
#include <opencv2/opencv.hpp>
#include <iostream>
int main() {
    std::cout << "OpenCV Version: " << CV_VERSION << std::endl;
#ifdef HAVE_CUDA
    std::cout << "CUDA is available." << std::endl;
#else
    std::cout << "CUDA is NOT available." << std::endl;
#endif
    return 0;
}
```

compile test_opencv.cpp：

```
g++ test_opencv.cpp -o test_opencv `pkg-config --cflags --libs opencv4` -I/usr/local/include/opencv4/opencv2
-L/usr/local/lib -Wl,-rpath,/usr/local/lib
```

run it

```
./test_opencv
```

If it displays the OpenCV version and "CUDA is available," it means OpenCV has successfully enabled CUDA support (see example image below).

2 Basic Knowledge of FFT and DFT

DFT is the mathematical formula, while FFT is the efficient algorithm for computing that formula; in practice, when we say "perform an FFT," we mean compute the DFT using a fast algorithm.

2.1 Introduction to DFT

DFT stands for **Discrete Fourier Transform**.
It is a mathematical tool that transforms a finite-length discrete-time sequence into a discrete-frequency sequence of the same length.

For a complex sequence (`x[0]`, `x[1]`, ..., `x[N-1]`) of length (N), the DFT is defined as:

$X[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-j\frac{2\pi}{N}kn}, \quad k = 0, 1, \ldots, N-1$

Where:

- `X[k]` is the k-th frequency component (frequency domain representation),
- j is the imaginary unit,
- N is the sequence length.

Using the **twiddle factor**: $W_N = e^{-j\frac{2\pi}{N}}$
The DFT can be written as: $X[k] = \sum_{n=0}^{N-1} x[n] \cdot W_N^{kn}$

Characteristics

- **Complexity**: Direct computation of the DFT formula requires (O(N^2)) complex multiplications (multiplying (N) times for each of the (N) values of (k)).
- **Accuracy**: The DFT is an exact, discrete, and finite mathematical definition—no approximations are involved.
- **Applications**: Signal processing, spectrum analysis, filter design, data compression, etc.

2.2 Introduction to FFT

FFT stands for **Fast Fourier Transform**.
It is **not a new transform**, but rather a **family of algorithms** that compute the DFT efficiently.

The FFT exploits the symmetry and periodicity of the exponential factors (twiddle factors ($e^{-j2\pi/N}$)) in the DFT formula. By recursively breaking down a DFT into smaller DFTs, the number of computations is drastically reduced.

Radix-2 FFT Decomposition

Using the **twiddle factor**: $W_N = e^{-j\frac{2\pi}{N}}$

When (N) is a power of 2, we split the sequence into **even-indexed** and **odd-indexed** samples.

Separate even and odd terms
$X[k] = \sum_{n=0}^{N/2-1} x[2n] W_N^{k(2n)} + \sum_{n=0}^{N/2-1} x[2n+1] W_N^{k(2n+1)}$

Using the identity:
$W_N^{2k} = W_{N/2}^k$

We obtain:
$X[k] = \sum_{n=0}^{N/2-1} x[2n] W_{N/2}^{kn} + W_N^k \sum_{n=0}^{N/2-1} x[2n+1] W_{N/2}^{kn}$

FFT is not a new transform — it efficiently computes the DFT using the recursive decomposition: $X[k] = E[k] + W_N^k O[k]$, reducing complexity from (O(N^2)) to (O(N \log N)).

Most Common FFT Algorithm

- **Cooley-Tukey Algorithm**: Requires (N) to be a power of 2 (radix-2 FFT).
- Basic idea: Decompose the DFT into two DFTs of length (N/2) (even-indexed and odd-indexed samples), and continue recursively.

Characteristics

- **Complexity**: Reduces from (O(N^2)) to (O(N \log N)).
- **Result**: Produces exactly the same numerical result as the DFT formula (within floating-point rounding errors).

- **Historical Impact**: The popularization of the FFT (Cooley and Tukey's 1965 paper) revolutionized digital signal processing, making real-time spectrum analysis practical.

2.3 Differences and Connections

Aspect	DFT	FFT
Nature	A mathematical transform definition	An efficient algorithm to compute the DFT
Computational Cost	(O(N^2))	(O(N \log N))
Result	Exact frequency domain representation	Numerically identical to the DFT result
Flexibility	Can be directly applied to any length (N)	Common algorithms require (N) to be a power of 2 (though algorithms for arbitrary lengths exist, e.g., Bluestein's algorithm)
Historical Emergence	Concept existed long ago (used by Gauss around 1805)	Modern efficient algorithm popularized in 1965

Key Connection:
The FFT is simply a fast way to compute the DFT. They are not two different transforms. One could say:

> "FFT is the high-speed algorithm that implements the DFT."

Think of it this way:

- **DFT** is like the **definition** of a task: "Calculate the sum of numbers from 1 to 100."
- The direct method would be: (1+2+3+...+100), adding step by step (similar to the O(N^2) approach).
- **FFT** is like the trick young Gauss discovered: ($(1+100) \times 100/2$), which drastically reduces the work (similar to the (O(N \log N)) improvement).

2.4 Practical Usage

In real-world programming (e.g., MATLAB, Python's `numpy.fft.fft`, C's FFTW library):

- When you call a function like `fft(x)`, you are asking for the **DFT** of (x).
- The library internally chooses the most appropriate fast algorithm (radix-2, mixed-radix, Bluestein's algorithm for prime lengths, etc.).
- Unless the sequence length is highly unusual (e.g., a large prime), your DFT will be computed using an **FFT algorithm**.

3 How to Apply 2D Discrete Fourier Filtering to a 2D Grayscale Image

Applying a **2D Discrete Fourier Transform (DFT)** for frequency-domain filtering follows this core pipeline:
Image → Frequency Domain → Multiply by Filter → Back to Image

The complete 2D frequency-domain filtering pipeline is:

1. Convert image to float.
2. (Optional) Multiply by ((-1)^{x+y}) to center spectrum.
3. Compute 2D DFT.
4. Design frequency mask (H(u,v)).
5. Multiply spectrum by mask.
6. Compute IDFT.
7. (If needed) Apply ((-1)^{x+y}) again.
8. Take real part.
9. Clip or normalize.
10. Save output image.

3.1 Treat the Grayscale Image as a 2D Signal

A grayscale image is simply a matrix: f(x,y)

with size $N \times M$.

Each pixel represents an intensity value (typically 0–255 or floating-point).
Mathematically, it is just a 2D discrete signal.

3.2 Compute the 2D DFT to Obtain the Frequency Spectrum F(u,v)

$F(u,v) = \sum_{x=0}^{N-1} \sum_{y=0}^{M-1} f(x,y), e^{-j2\pi(\frac{ux}{N} + \frac{vy}{M})}$

Important properties:

- F(u,v) is **complex-valued** (has magnitude and phase).
- The **magnitude** tells how strong a frequency component is.
- The **phase** encodes structural and spatial information.
- Phase is extremely important — removing it destroys image structure.

3.3 Spectrum Centering (Shift) is Important

By default, the DC component (low frequency) appears in the corner of the spectrum.
For filter design (especially circular masks), it is much easier if the low frequency is at the center.
There are two equivalent common approaches:

A) Frequency-domain shift (Quadrant swap)
Swap the four quadrants of the spectrum so that the low frequency moves to the center.

B) Spatial-domain pre-multiplication by ((-1)^{x+y})

Before computing the DFT, multiply the image by: (-1)^{x+y}

This is mathematically equivalent to shifting the spectrum to the center.
If you use this approach, you must apply the same operation again after inverse transform to restore the spatial result.
You must remain consistent — do not double-shift.

3.4 Design the Frequency-Domain Filter H(u,v)

A frequency filter is simply a mask with the same size as the spectrum.
Filtering is done by pointwise multiplication: $G(u,v) = F(u,v) \cdot H(u,v)$

Common filter types:

1 Low-pass Filter (Blur / Denoise)

- Keeps low frequencies (center region).
- Suppresses high frequencies (details, noise).
- Produces smoothing or blur.
  Example:
- Gaussian low-pass (preferred because it reduces ringing)

---

2 High-pass Filter (Edge Enhancement)

- Suppresses low frequencies.
- Keeps high frequencies.
- Enhances edges and fine details.

Often defined as:
$H_{hp} = 1 - H_{lp}$

3
Band-pass:

- Keeps a specific frequency range.
- Enhances textures.

Band-stop:

- Removes a specific frequency band.
- Useful for removing structured noise patterns.

---

4 Notch Filter
Periodic noise in images appears as **pairs of bright symmetric points** in the spectrum.
A notch filter removes those specific frequency locations.
Used for:

- Removing stripe noise
- Removing periodic interference

3.5 Apply the Inverse DFT (IDFT)

After filtering:

$g(x,y)=\frac{1}{NM}\sum*{u=0}^{N-1}\sum*{v=0}^{M-1} G(u,v),e^{j2\pi(\frac{ux}{N}+\frac{vy}{M})} $
Important:

- The inverse transform requires normalization by (1/(NM)).
- Some implementations apply scaling in forward transform instead.
- Be consistent with your normalization strategy.

If you used ((-1)^{x+y}) before the DFT, you must multiply again after IDFT to restore the image.

3.6 Output Processing

Due to numerical precision:

- The imaginary part after IDFT should be nearly zero.
- Use the real part of the result.

Then:

- Clip values to valid intensity range (e.g., 0–255).
- Or normalize appropriately.

3.7 How to Visualize the Spectrum

Raw magnitude values span a very large dynamic range.

To visualize:
$S(u,v) = \log(1 + |F(u,v)|)$

Then normalize to 0–255 and save as a grayscale image.
Without logarithmic scaling, the spectrum will look mostly black except for a few bright pixels.

4 Implementation

4.1 What this program does

This is a 2D DFT-based or 2D FFT-based image filtering on Linux:

1. Generate synthetic **N×N grayscale** images (N=2048, A–F case each case has its own input image).)
2. compute DFT or FFT
    1. Compute **2D DFT**(complex spectrum) using separable 1D DFT (O(N³), still true DFT) or
    2. Perform a 2D FFT on the input image (with shift=true centering). using separable 1D FFT
3. Apply frequency-domain filters (low-pass, high-pass, band-pass, band-stop , auto-notch.)
4. Compute **2D IDFT or 2D IFFT** back to spatial domain
5. Save
    1. output images as PGM
    2. For every saved image, also save a **spectrum visualization image** (magnitude → log scale → normalized to 0..255) as another `.pgm`
- `performance.csv` : average timings for stages like FFT2, mask, IFFT2, etc.

- `metrics.csv` : per-image hash (u8 + f64) and (if a reference image exists) error statistics (L2/RMSE/max_abs/PSNR) a reference

4.2 How to run and what arguments mean

```
./dft_benchmark_cpu N runs outdir
```

- `N` : image size (must be positive and even)
- `runs` : how many times to repeat timed stages (DFT/mask/IDFT) and average
- `outdir` : output directory for images and CSV files

4.3 Core data types and utilities

Timing: `Timer` and `avg_ms()`

- `Timer` measures elapsed time in milliseconds.
- `avg_ms(runs, fn)` runs `fn()` multiple times and returns the average time.
- This is how the code reports stable timings for CUDA comparisons later.

Image container: `struct Image`

- Stores:
  - `w, h` : dimensions
  - `px` : `std::vector<double>` pixels (grayscale)
- Using `double` is important because IDFT results can be non-integer and can go negative.
- The image is grayscale; pixels are stored as double.
- When writing to disk, write_pgm() clamps pixels to 0..255, converts to uint8, and writes P5 PGM.
- normalize_to_0_255() is used to map certain results (e.g., high-pass edges, band-pass texture) into a visible range (otherwise values may be negative or have too large a range).

Saving images: `write_pgm()`

- Writes **PGM P5** (binary) grayscale format.
- Pixel values are:
  - clamped to `[0,255]`
  - rounded to `uint8_t`
- This is the exact byte representation that `hash_u8` corresponds to.

Normalization: `normalize_to_0_255()`
High-pass and band-pass IDFT outputs are typically:

- centered around 0,
- can have negative values,
- may not lie in `[0,255]` .
  So for visualization, the code does min-max scaling to `[0,255]` .
  That's why B and C store "*_norm.pgm".

4.4 Spectrum visualization

1 `spectrum_to_image(F, w, h, shift_center)`
Purpose: turn a complex spectrum into a viewable grayscale image.

Steps:

1. Compute magnitude: `|F(u,v)|`
2. Log scale: `log(1 + |F|)` (so huge DC doesn't dominate)
3. Normalize by max value → map to `[0,255]`

2 The key "shift" detail (important)
Your comment is correct:

- You call `dft2(..., shift=true)`
  That multiplies the input image by `(-1)^(x+y)` before DFT.
- This **already centers DC/low frequencies** in the resulting spectrum `F`.
- Therefore, **you should NOT do another quadrant swap** when visualizing the spectrum.

That's why spectrum saving uses:

```
spectrum_to_image(F, N, N, /*shift_center=*/false)
```

If you mistakenly set `shift_center=true` here, you would "re-shift" and the spectrum would look wrong (DC would move away from center again).

4.5 saving spectrum for each saved image

1 `save_spectrum_from_F(fname, F)`

- Converts an already-computed spectrum `F` into an image and saves it.
- Uses `shift_center=false` (because DFT already centered).
- Records metrics under a "spectrum-like" entry.

Important subtlety: This does **not** measure performance. It's just output generation.

---

2 `save_spectrum_from_img(test, tag, img)`

- Computes `F = dft2(img, ...)` internally
- Saves spectrum image named:
  `test + "_" + tag + "_spectrum.pgm"`
- Records metrics for that spectrum image too.

Important subtlety: This extra DFT is explicitly not included in timed performance.csv stages
So: total runtime will increase, but your benchmark timing numbers remain comparable.

4.6 DFT/IDFT implementation in CPU

1 `Twiddle1D`

Precomputes twiddle factors:

- Forward: $W[k,n] = e^{-i2\pi kn/N}$
- Inverse: $W^{-1}[k,n] = e^{+i2\pi kn/N}$

This avoids recalculating sin/cos in the hottest loops.

---

2 `dft1d()`

Computes:
$out[k] = \sum_{n=0}^{N-1} in[n] \cdot table[k,n]$

---

3 `dft2(img, shift=true)`

Computes 2D DFT by separability:

1. For each row `y` : do a 1D DFT → store into `temp(u, y)`
2. For each column `u` : do a 1D DFT → store into `F(u, v)`

`shift=true` means centering the spectrum

It multiplies spatial samples by `(−1)^(x+y)`, which shifts the spectrum so low frequencies land in the center.

`idft2(F, shift=true)`

Performs the inverse in two stages:

1. inverse DFT along columns
2. inverse DFT along rows
3. divide by `N*N` (normalization)
4. undo the shift with `(−1)^(x+y)` if enabled

4.7 FFT/IFFT implementation in CPU

4.7.1 fft1d_inplace()

This is a standard iterative radix-2 Cooley–Tukey FFT:

(1) Bit-reversal permutation

```
for (i=1..n−1) { update j; if(i<j) swap(a[i],a[j]); }
```

FFT butterfly stages assume inputs are arranged in bit-reversed order, so the code performs this reordering first.

(2) Layer-by-layer butterflies

```
for (len=2; len<=n; len<<=1) {
  wlen = exp(±i 2π/len)
  for (each block) {
    for (j< len/2) {
      u=a[i+j]
      v=a[i+j+half]*w
      a[i+j]=u+v
      a[i+j+half]=u−v
      w*=wlen
    }
  }
}
```

inverse=false uses -i (forward FFT).
inverse=true uses +i (inverse IFFT).

(3) Normalization for IFFT

```
if (inverse) for i a[i] *= 1/n;
```

So for the 2D IFFT:

- Inverse-transform columns: each column divides by N.
- Inverse-transform rows: each row divides by N again.
  - The combined factor is 1/(N*N), matching your DFT version where idft2 multiplies by inv = 1/(N*N) at the end.

4.7.2 2D FFT: fft2()

`std::vector<cd> fft2(const Image& img, bool shift=true)`
It is separable (the same strategy as your DFT version):

(1) Input shift (center DC)

```
double f = img(x,y);
if (shift && ((x+y)&1)) f = -f;
row[x] = (f,0)
```

Multiplying by (-1)^(x+y) in the spatial domain corresponds to shifting the spectrum, placing DC at the center.

(2) Row FFT
Apply 1D FFT on each row and store results as F(u,y).

(3) Column FFT
Apply 1D FFT on each column and write back to F(u,v).
The output F is a N*N complex array with the same memory layout as the DFT version: id(x,y)=y*N+x.

4.7.3 2D IFFT: ifft2()

`Image ifft2(const std::vector<cd>& F, int N, bool shift=true)`

This is the reverse direction:

1. Copy tmp=F.
2. Apply fft1d_inplace(col,true) on each column (includes 1/N normalization).
3. Apply fft1d_inplace(row,true) on each row (includes another 1/N normalization).
4. Take the real part as the output pixel value.
5. Apply (-1)^(x+y) again to undo the shift and return to the normal spatial layout.

Note: Like your DFT version, it assumes the input image is real-valued, so it uses only .real() for the final image.

4.8 Synthetic input generation

1 `make_scene_mix(N)`

Creates a "rich" test image:

- smooth gradient (low frequency)
- bright rectangle (strong edges)
- ring (structured edge content)
- diagonal line (thin high-frequency detail)

This is a good general-purpose CUDA comparison input.

---

2 `make_checkerboard(N, block)`
High-frequency stress test. Great to verify filtering correctness.

---

3 `make_circles(N)`
Radial sinusoidal rings, useful to test band-stop removal.

---

4 `add_periodic_noise_multi(src, amp, ks)`

Adds multiple sinusoidal patterns:

- creates distinct spikes in frequency domain
- ideal for notch filtering experiments

4.9 Frequency-domain masks (filters) and apply_mask()

Mask generation functions reuse exactly the same logic as your DFT version:

- gaussian_lowpass(N, sigma): high in the center, low outside.
  - Produces a smooth blur by preserving low frequencies near the center and attenuating high-frequency details.
- gaussian_highpass: 1 - lowpass.
  - Enhances edges and fine details by suppressing low frequencies and keeping high-frequency components.
- ideal_bandpass(r0,r1): 1 inside the radius interval, 0 outside.
  - Preserves structures within a specific frequency range, emphasizing textures or patterns of a certain scale.
- ideal_bandstop: 1 - bandpass.
  - Removes structures within a specific frequency range, suppressing periodic patterns or unwanted frequency components.
- auto_notch_mask():
  - detect "abnormal spikes" and zero out small disks around them.
  - automatically detects and suppresses strong periodic noise frequencies in the spectrum while preserving the rest of the signal.

apply_mask(F,H) is elementwise multiplication:

```
G[i] = F[i] * H[i];
```

4.9.1 Gaussian low-pass

`gaussian_lowpass` : smooth, reduces ringing compared to ideal low-pass

In the frequency domain, the Gaussian low-pass filter is defined as:

$H(u,v) = e^{-\frac{D(u,v)^2}{2\sigma^2}}$

Where:

- $D(u,v)$ is the distance from the frequency point (u,v) to the center of the spectrum (the DC component).
- σ controls the strength of the filtering:
  - Small $\sigma \rightarrow$ stronger blur (narrower passband)
  - Large $\sigma \rightarrow$ weaker blur (wider passband)

CPU Version

```
std::vector<double> gaussian_lowpass(int N, double sigma)
{
    // Allocate N x N frequency mask (row-major layout)
    std::vector<double> H((size_t)N*N, 0.0);

    // Center of frequency domain (DC component location)
    int cx = N / 2;
    int cy = N / 2;

    // Helper to convert 2D index (x,y) to 1D index
    auto id = [&](int x, int y) {
        return (size_t)y * N + x;
    };

    // Precompute denominator term: 2*sigma^2
    // Add small epsilon to avoid division by zero if sigma is extremely small
    double two_sigma2 = 2.0 * sigma * sigma + 1e-12;

    // Loop over all frequency coordinates
    for (int y = 0; y < N; ++y)
    {
        for (int x = 0; x < N; ++x)
        {
            // Distance from current frequency point to spectrum center
            double du = x - cx;
            double dv = y - cy;

            // Squared distance
            double d2 = du * du + dv * dv;

            // Gaussian response
            H[id(x, y)] = std::exp(-d2 / two_sigma2);
        }
```

```
    }

    return H;
}
```

4.9.2 Gaussian high-pass

`gaussian_highpass = 1 - gaussian_lowpass`

Enhances edges and fine details by suppressing low frequencies and keeping high-frequency components.

CPU Version

```
std::vector<double> gaussian_highpass(int N, double sigma) {
    auto glp = gaussian_lowpass(N, sigma);
    for(auto& v : glp) v = 1.0 - v;
    return glp;
}
```

4.9.3 Important Note (Ideal filter vs Gaussian filter)

This is an ideal filter

- That means:
  - Sharp discontinuities in frequency domain (hard cutoff 0->1)
  - Causes ringing artifacts (Gibbs phenomenon)
  - Not smooth

A Gaussian band-pass would avoid ringing because it transitions smoothly instead of abruptly.

Ringing refers to an artifact in images or signals where ripple-like or oscillatory patterns appear near sharp edges or sudden intensity transitions.
In simple terms: What should be a "clean edge" instead shows circular bright and dark ripples around it, similar to water waves spreading outward.

---

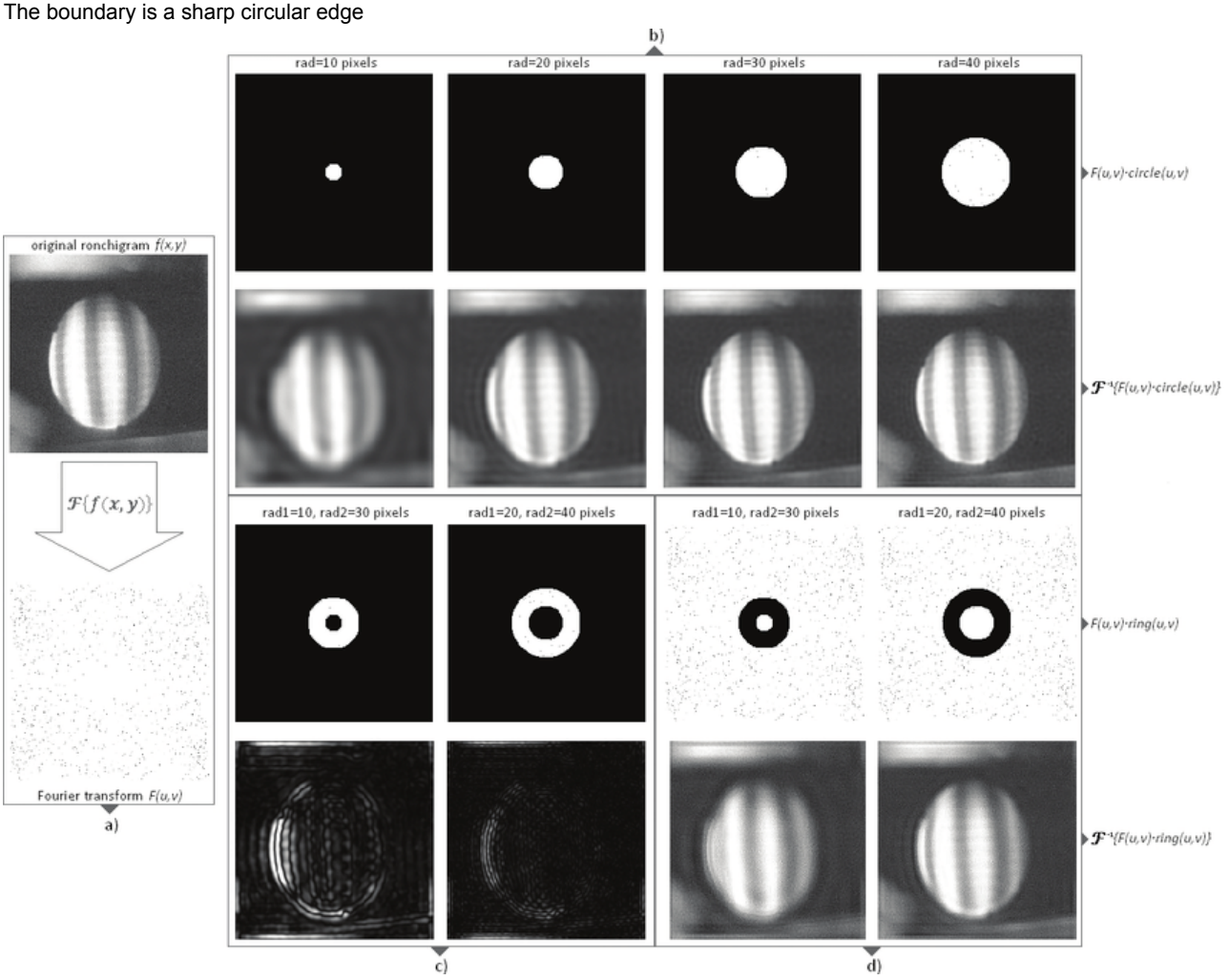
hard cutoff

- In the frequency domain, a frequency threshold is applied with a sudden truncation. All frequencies inside the threshold are fullpreserved, and all frequencies outside the threshold are set to 0. There is no transition region.

What Does Hard Cutoff Look Like in the Frequency Domain?

- From the images, you can observe:
- The center represents low frequencies
- Inside radius r → value = 1 (completely preserved)
- Outside radius r → value = 0 (completely removed)

The boundary is a sharp circular edge



4.9.4 Ideal band-pass

Preserves structures within a specific frequency range, emphasizing textures or patterns of a certain scale.

band-pass keeps only frequencies in `[r0, r1]`

In frequency space, this filter looks like a ring:

```
        High freq
     ***************
  ****           ****
***     Pass        ***
**       Band          **
***                   ***
  ****               ****
     ***************
        Low freq
```

The center (low frequencies) is removed.
The outer region (very high frequencies) is removed.
Only a middle circular band remains.

When applied in frequency domain:

- Removes smooth background (low frequencies)
- Removes very fine noise (high frequencies)
- Emphasizes texture and structural details

Good for:

- Texture enhancement
- Feature extraction
- Frequency analysis experiments

4.9.4.1 mathematical interpretation

This function creates an **Ideal Band-Pass Filter (IBPF)** in the frequency domain.

It generates an (N \times N) frequency mask that:

- Keeps only frequencies within a specific radial range ([r_0, r_1])
- Suppresses everything else

In other words, it preserves a ring of frequencies and blocks both low and high frequencies.

The ideal band-pass filter is defined as:

$H(u,v) =$

- 1, if $r_0 \leq D(u,v) \leq r_1$
- 0, if otherwise

Where:

- (D(u,v)) is the distance from frequency coordinate ((u,v)) to the center of the spectrum
- (r_0) = inner radius
- (r_1) = outer radius

So

- Frequencies below (r_0) → removed (low frequencies removed)
- Frequencies above (r_1) → removed (high frequencies removed)
- Only middle frequencies survive

This creates a circular "ring" in frequency space.

4.9.4.2 Code

```
std::vector<double> ideal_bandpass(int N, double r0, double r1) {
    std::vector<double> H((size_t)N*N, 0.0);
    int cx=N/2, cy=N/2;
    auto id = [&](int x,int y){ return (size_t)y*N + x; };
    for(int y=0;y<N;++y){
        for(int x=0;x<N;++x){
            double du=x-cx, dv=y-cy;
            double d=std::sqrt(du*du+dv*dv);
            H[id(x,y)] = (d>=r0 && d<=r1) ? 1.0 : 0.0;
        }
    }
    return H;
}
```

4.9.5 Ideal band-stop

Removes structures within a specific frequency range, suppressing periodic patterns or unwanted frequency components.

band-stop removes only frequencies in `[r0, r1]`

CPU Version

```
std::vector<double> ideal_bandstop(int N, double r0, double r1) {
    auto bp = ideal_bandpass(N, r0, r1);
    for(auto& v : bp) v = 1.0 - v;
    return bp;
}
```

4.9.6 `apply_mask`

apply_mask(F,H) is elementwise multiplication:

```
std::vector<cd> apply_mask(const std::vector<cd>& F, const std::vector<double>& H) {
    std::vector<cd> G(F.size());
    for(size_t i=0;i<F.size();++i) G[i] = F[i] * H[i];
    return G;
}
```

4.9.7 Auto notch mask

Goal: detect periodic noise spikes automatically.
automatically detects and suppresses strong periodic noise frequencies in the spectrum while preserving the rest of the signal.

Method:

1. Compute magnitudes `|F|`
2. Estimate a robust baseline via the **median magnitude** `median`
3. Scan frequency bins with radius greater than `center_keep` from the center (do not touch the center region).
    1. Ignore a center disk (`center_keep`) so you don't delete the low-frequency content
4. If a frequency bin's magnitude exceeds `threshold_factor * median` , treat it as a spike
5. Zero out a small disk around it (`notch_radius`) that spike,
6. Then `apply_mask()` multiplies `F * H` in frequency domain.

This matches your requirement "only use auto notch for denoising": no manual specification of noise locations.

```
std::vector<double> auto_notch_mask(const std::vector<cd>& F, int N,
                                    double center_keep=24.0,
                                    double threshold_factor=12.0,
                                    int notch_radius=3) {
    std::vector<double> H((size_t)N*N, 1.0);
    int cx=N/2, cy=N/2;
    auto id = [&](int x,int y){ return (size_t)y*N + x; };

    std::vector<double> mags;
    mags.reserve((size_t)N*N);
    for(const auto& v : F) mags.push_back(std::abs(v));
    std::nth_element(mags.begin(), mags.begin()+mags.size()/2, mags.end());
    double med = mags[mags.size()/2] + 1e-12;

    for(int y=0;y<N;++y){
        for(int x=0;x<N;++x){
            double du=x-cx, dv=y-cy;
            double d=std::sqrt(du*du+dv*dv);
            if (d <= center_keep) continue;

            double m = std::abs(F[id(x,y)]);
            if (m > threshold_factor * med) {
                for(int yy=y-notch_radius; yy<=y+notch_radius; ++yy){
                    for(int xx=x-notch_radius; xx<=x+notch_radius; ++xx){
                        if(xx<0||xx>=N||yy<0||yy>=N) continue;
                        double rr = std::sqrt((xx-x)*(xx-x) + (yy-y)*(yy-y));
                        if (rr <= notch_radius) H[id(xx,yy)] = 0.0;
                    }
                }
            }
        }
    }
    return H;
}
```

4.10 Metrics: hashes + error measures

1 Hashing

Two hashes per image:

1. `hash_u8`
    1. clamp to uint8 and run FNV-1a 64-bit hash byte-by-byte (useful for checking "final output file-level equality").
    2. Hashes the bytes that would be written to the PGM (after clamp+round).
    3. This is usually the most practical for CUDA "same output image" checking.
2. `hash_f64`
    1. run FNV-1a 64-bit hash over the raw bytes of the double pixel array (very sensitive; even tiny floating-point differences will change it).
    2. Hashes raw double bytes. Very strict, often differs with small floating-point changes.

Both use 64-bit FNV-1a.

---

2 Error stats (optional)

When you pass a reference image, it computes:

- `L2` = sqrt(sum (a-b)^2)
- `RMSE` = sqrt(mean (a-b)^2)
- `MaxAbs` = max |a-b|
- `PSNR` (assuming range 0..255) = 10 log10(255^2 / MSE) (if MSE ≈ 0 then PSNR is infinity)

These are written to `metrics.csv`.

4.11 Performance logging: `performance.csv`

For each test case, it records stages like:

- `DFT2`
- `Mask_*`
- `IDFT2`
- plus `Build_auto_notch` for test E

Each stage time is averaged over `runs`.
avg_ms(runs, fn) runs the stage runs times and records the average elapsed time.

Note: It used solely to generate spectrum images is not included in performance timing, because save_spectrum_from_img computes a spectrum only for output purposes.

4.12 What each test case A–F produces (including spectrum outputs)

- A Gaussian low-pass blur: scene_mix → FFT → gaussian lowpass → IFFT → blur
- B Gaussian high-pass edges: scene_mix → highpass → IFFT → edges → normalize for visualization
- C Band-pass texture emphasis: scene_mix → ideal bandpass → IFFT → normalize
- D Checkerboard stress: checkerboard → lowpass → IFFT (tests behavior under strong high-frequency input)
- E Periodic noise + auto notch: base_clean + sinusoidal noise → auto_notch → denoised
- F Radial rings + band-stop: rings image → ideal bandstop → IFFT (remove a certain radius range of frequencies)

For each test, you save:

- the input image
- the output image(s)
- **and now also spectrum images** for those saved images

---

1 Gaussian low-pass blur

- Saves:
    - `A..._0_src.pgm`
    - `A..._0_src_spectrum.pgm` (computed separately)
    - `A..._0_src_spectrum_fromF.pgm` (from the timed DFT result)

- `A..._1_blur.pgm`
- `A..._1_blur_spectrum.pgm`

---

2 Gaussian high-pass edges

- Saves:
  - source + source spectrum
  - edges output is normalized: `..._1_edges_norm.pgm`
  - spectrum of the normalized edges image

---

3 Band-pass texture emphasis

- Saves:
  - source + source spectrum
  - band-pass output normalized: `..._1_bandpass_norm.pgm`
  - spectrum of that normalized result

---

4 Checkerboard

- Saves:
  - checkerboard + spectrum
  - low-pass result + spectrum

---

5 Periodic noise → auto notch

- Saves:
  - base clean + spectrum
  - noisy + spectrum
  - denoised + spectrum
  - also saves `..._spectrum_fromF.pgm` for the noisy DFT already computed in timing

---

6 Radial rings → band-stop

- Saves:
  - rings source + spectrum
  - band-stop output + spectrum

4.13 Outputs summary

In `outdir` you will get:

- Many `.pgm` images:
  - `*_src.pgm`, `*_result.pgm`, and `*_spectrum.pgm`
- `performance.csv`: timed stages only
- `metrics.csv`: hash + optional error stats for:
  - normal images
  - spectrum images (usually with no reference)

5 Optimization using CUDA techniques

> The DFT and FFT algorithms in the chapters above are optimized using the same common CUDA optimization techniques. Next, I will use the DFT as an example to explain how the optimization is done.

Key points (GPU vs CPU)

- Converting column DFT into row DFT via transpose to ensure coalesced memory access
- Using shared-memory tiled transpose with padding to avoid bank conflicts
- Computing twiddle factors via recurrence instead of loading large tables
- Using `__restrict__` to enable better compiler optimization
- Fusing scaling and inverse shift into the final kernel to reduce global memory traffic and kernel launches

The GPU is still **O(N³)** overall (same complexity class as CPU DFT, just massively parallel).

5.1 Pipeline comparison (CPU vs GPU)

1 CPU pipeline (conceptual)

```
F = dft2(img, tw, shift=true);          // row DFT + column DFT (strided column gather)
G = apply_mask(F, H);                   // multiply on CPU
out = idft2(G, tw, shift=true);         // inverse col + inverse row, then scale+invshift on CPU
```

---

2 GPU pipeline (conceptual)

```
img_to_complex_shift<<<...>>>(...);     // shift + convert

dft_rows_recurrence<<<...>>>(...);      // row DFT (coalesced)
transpose_cdouble2<<<...>>>(...);       // tiled transpose
dft_rows_recurrence<<<...>>>(...);      // row DFT on transposed data (coalesced)
transpose_cdouble2<<<...>>>(...);       // transpose back

apply_mask_kernel<<<...>>>(...);        // mask multiply on GPU

transpose + row IDFT + transpose + row IDFT
complex_to_img_scale_invshift<<<...>>>  // fused scale+invshift
```

5.2 Baseline difference: CPU uses precomputed twiddle table; GPU computes twiddle on the fly

1 CPU (precompute full twiddle table `N*N`)

CPU builds `Twiddle1D` storing `W[k*N + n]` and `Winv[k*N + n]` :

```
struct Twiddle1D {
    int N;
    std::vector<cd> W;    // forward: exp(-i2pi kn/N)
    std::vector<cd> Winv; // inverse: exp(+i2pi kn/N)
    explicit Twiddle1D(int n): N(n), W((size_t)n*n), Winv((size_t)n*n) {
        for(int k=0;k<N;++k){
            for(int n0=0;n0<N;++n0){
                double ang = 2.0*PI*(double)k*(double)n0/(double)N;
                W[(size_t)k*N + n0]    = cd(std::cos(-ang), std::sin(-ang));
                Winv[(size_t)k*N + n0] = cd(std::cos(+ang), std::sin(+ang));
            }
        }
    }
};
```

CPU's 1D DFT then multiplies input by `row[n]` read from that table:

```
void dft1d(const cd* in, cd* out, int N, const std::vector<cd>& table) {
    for (int k=0;k<N;++k){
        cd s(0,0);
        const cd* row = &table[(size_t)k*N];
        for (int n=0;n<N;++n) s += in[n] * row[n];
        out[k] = s;
```

```
    }
}
```

---

2 GPU (no twiddle table; sincos + recurrence)

GPU computes one `w_step` per output and iterates `w *= w_step`:

```
double sign = inverse ? +1.0 : -1.0;
double ang_step = sign * (2.0 * PI * (double)k / (double)n);

double sn, cs;
sincos(ang_step, &sn, &cs);

cdouble2 w_step{cs, sn};
cdouble2 w{1.0, 0.0};
for(int x=0; x<n; ++x){
    sum = cadd(sum, cmul(in[base + x], w));
    w = cmul(w, w_step);
}
```

Why this matters:

- CPU: extra memory is fine; caches help; table reduces trig calls.
- GPU: a full twiddle table is **huge bandwidth pressure**. Recurrence is a standard tradeoff: **more FLOPs, much less global memory traffic**.

5.3 Optimization 1: Transpose-based 2D DFT to avoid strided column access

After doing "row DFT", you must do "column DFT".

- On CPU: column access is strided too, but caches and scalar execution handle it reasonably.
- On GPU: strided column reads/writes are typically **not coalesced**, which is very slow.

1 CPU approach (direct column DFT without transpose)

CPU does row DFT into `temp`, then column DFT by gathering a column vector:

```
// Row DFT
for(int y=0;y<N;++y){
    ... fill inrow ...
    dft1d(inrow.data(), outrow.data(), N, tw.W);
    for(int u=0;u<N;++u) temp[id(u,y)] = outrow[u];
}

// Col DFT (strided access to temp)
for(int u=0;u<N;++u){
    for(int y=0;y<N;++y) incol[y] = temp[id(u,y)];
    dft1d(incol.data(), outcol.data(), N, tw.W);
    for(int v=0;v<N;++v) F[id(u,v)] = outcol[v];
}
```

---

2 approach (transpose so both passes are row-contiguous)

GPU performs:

RowDFT → Transpose → RowDFT → Transpose back

```
// Row DFT pass #1
dft_rows_recurrence<<<grd_dft, blk_dft, 0, stream>>>(d_c0, d_c1, n, 0);

// Transpose
```

```
transpose_cdouble2<32,8><<<grdT, blkT, 0, stream>>>(d_c1, d_c2, n);

// Row DFT pass #2 (on transposed data == column DFT)
dft_rows_recurrence<<<grd_dft, blk_dft, 0, stream>>>(d_c2, d_c1, n, 0);

// Transpose back
transpose_cdouble2<32,8><<<grdT, blkT, 0, stream>>>(d_c1, d_c0, n);
```

Why this is a CUDA optimization:

- Both DFT passes read contiguous memory (rows), enabling **coalesced global loads** across warps.

5.4 Optimization 2: Tiled shared-memory transpose + padding to reduce bank conflicts

CPU transpose?
CPU version doesn't need a transpose kernel at all (it just gathers columns).

---

3 GPU transpose kernel (classic NVIDIA pattern)

Key idea: stage a `32x32` tile in shared memory, with padding `+1`:

```
template<int TILE_DIM, int BLOCK_ROWS>
__global__ void transpose_cdouble2(const cdouble2* __restrict__ in,
                                   cdouble2* __restrict__ out,
                                   int n)
{
    __shared__ cdouble2 tile[TILE_DIM][TILE_DIM+1]; // +1 padding

    int x = blockIdx.x * TILE_DIM + threadIdx.x;
    int y = blockIdx.y * TILE_DIM + threadIdx.y;

    // coalesced read into shared
    for (int i=0; i<TILE_DIM; i+=BLOCK_ROWS){
        int yy = y + i;
        if (x < n && yy < n) tile[threadIdx.y + i][threadIdx.x] = in[yy*n + x];
    }
    __syncthreads();

    // coalesced write from shared (transposed)
    int ox = blockIdx.y * TILE_DIM + threadIdx.x;
    int oy = blockIdx.x * TILE_DIM + threadIdx.y;
    for (int i=0; i<TILE_DIM; i+=BLOCK_ROWS){
        int oyy = oy + i;
        if (ox < n && oyy < n) out[oyy*n + ox] = tile[threadIdx.x][threadIdx.y + i];
    }
}
```

Why it's fast on GPU:

- Global reads and writes are both **coalesced**.
- Shared memory padding `TILE_DIM+1` reduces **bank conflicts** for transposed access.

5.5 Optimization 3: Twiddle recurrence + `sincos()` (reduce memory traffic)

1 CPU (table lookup, no trig inside main loop)

CPU uses precomputed `tw.W` / `tw.Winv`:

```
const cd* row = &table[(size_t)k*N];
for (int n=0;n<N;++n) s += in[n] * row[n];
```

---

2 GPU (compute once, then multiply recurrence)

```
double sn, cs;
sincos(ang_step, &sn, &cs);
cdouble2 w_step{cs, sn};
cdouble2 w{1.0, 0.0};

for(int x=0; x<n; ++x){
    sum = cadd(sum, cmul(a, w));
    w = cmul(w, w_step);
}
```

Comparison summary:

- CPU: **more memory, fewer FLOPs** (table loads).
- GPU: **more FLOPs, less global memory** (often better on GPU).

5.6 Optimization : `__restrict__` pointers (help compiler optimize memory accesses)

1
CPU functions do not use `restrict` (C++ doesn't commonly apply it here).

---

2 GPU kernels mark pointers `__restrict__`

Example:

```
__global__ void apply_mask_kernel(const cdouble2* __restrict__ F,
                                  const double* __restrict__ H,
                                  cdouble2* __restrict__ G,
                                  int npx)
```

Tells the compiler that pointers do **not alias**, enabling better instruction scheduling and fewer redundant loads.

5.7 Optimization 5: Fuse scale + inverse shift into the final output kernel

1 CPU (final scaling + inverse shift is done in a loop on host)

CPU `idft2` ends with:

```
double inv = 1.0 / (double)(N*N);
for(int y=0;y<N;++y){
    for(int x=0;x<N;++x){
        double f = out[id(x,y)].real() * inv;
        if (shift && ((x+y)&1)) f = -f;
        img.at(x,y) = f;
    }
}
```

GPU (fused into final kernel)

Instead of separate passes, GPU does everything in one kernel:

```
__global__ void complex_to_img_scale_invshift(const cdouble2* __restrict__ in,
                                              double* __restrict__ img,
                                              int n, int shift_on)
{
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;
    if (x>=n || y>=n) return;

    int idx = y*n + x;
    double inv = 1.0 / (double)(n*n);
```

```
    double f = in[idx].x * inv;          // real + scale
    if (shift_on && ((x+y)&1)) f = -f;  // undo shift
    img[idx] = f;
}
```

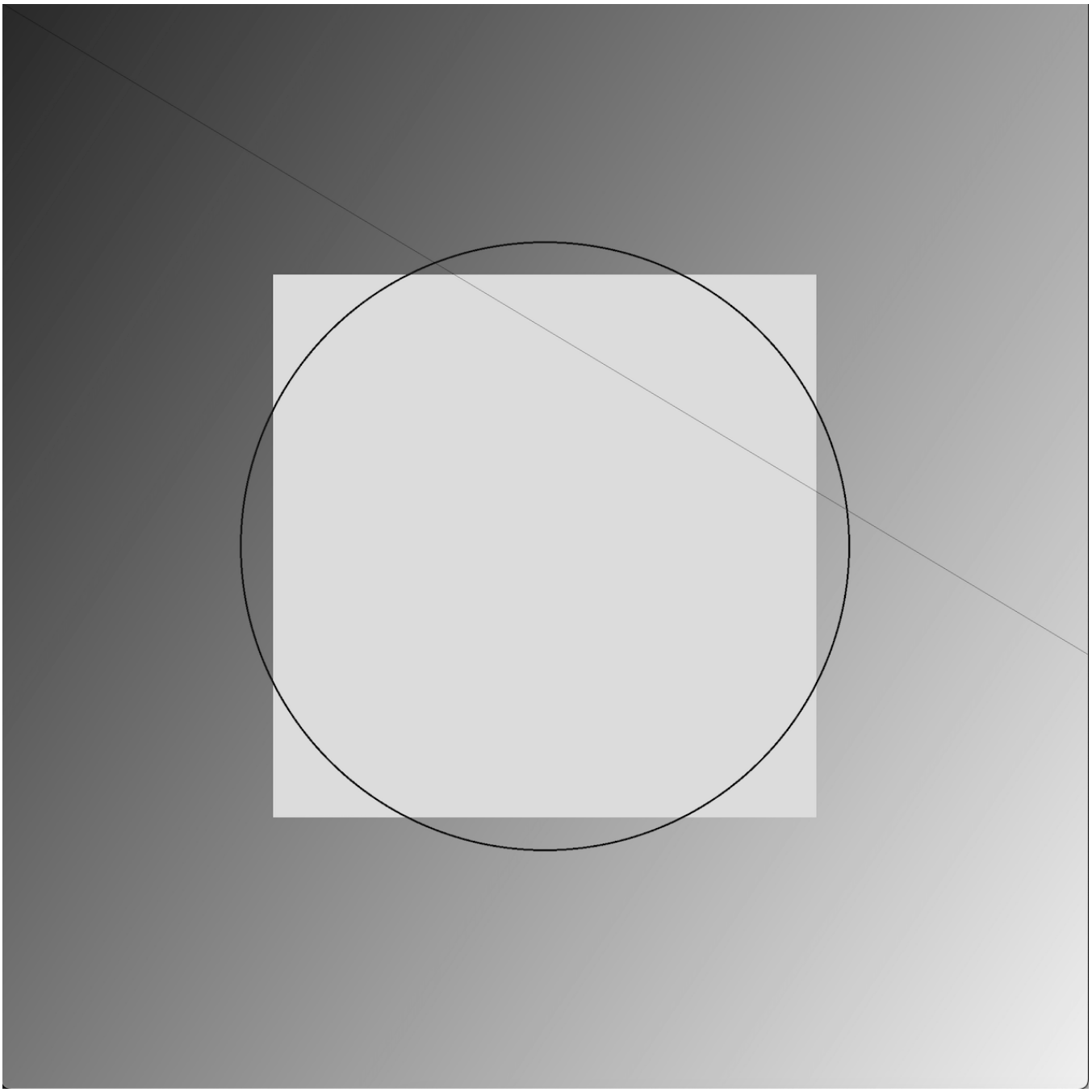
Why it's a CUDA optimization:

- Fewer kernel launches.
- Less global memory traffic (no intermediate buffers for scale/shift).

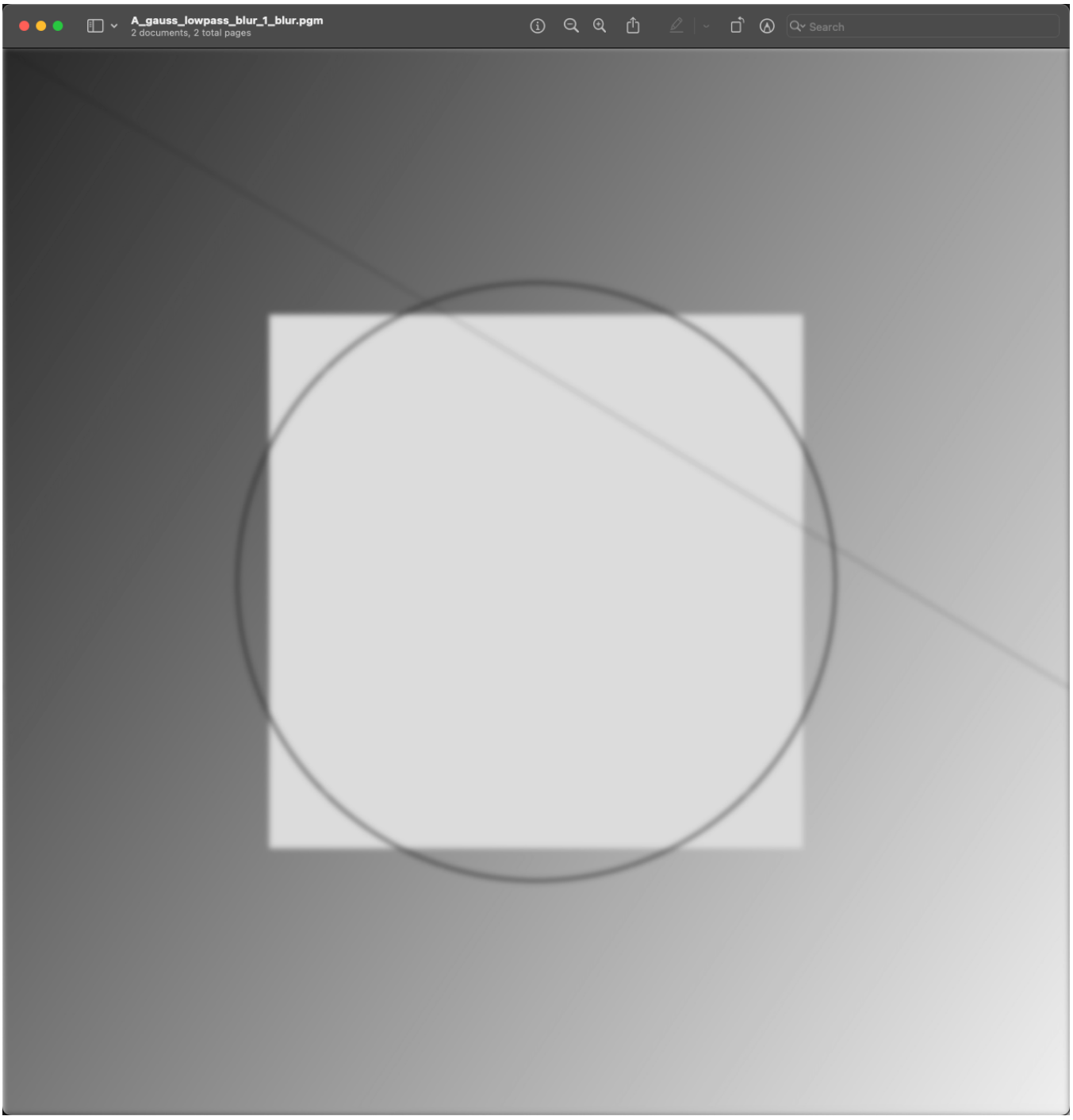
6 Result

6.1 image (2048x2048 pixel)

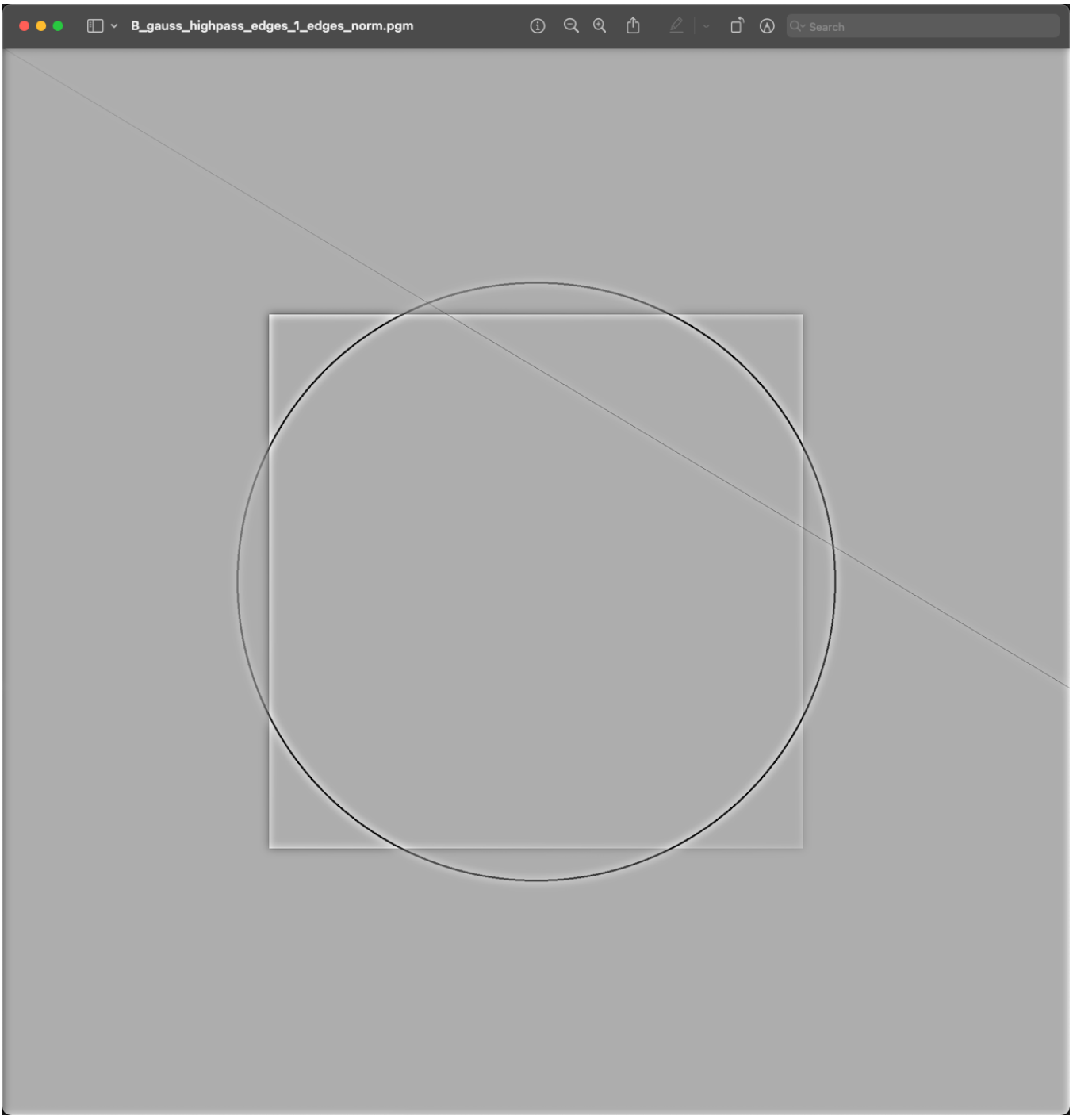
6.1.1 Original Image for Testcase A,B,C



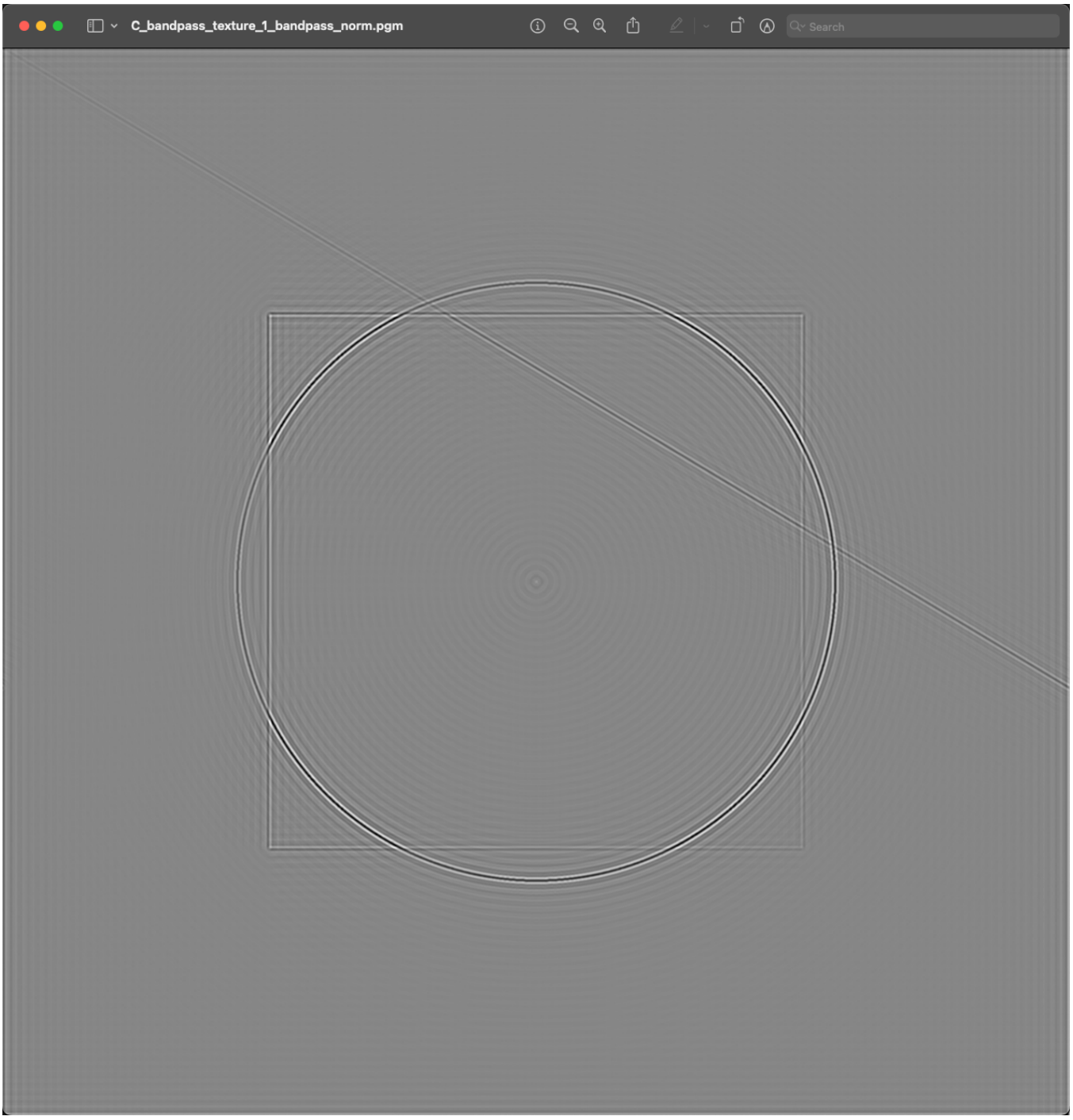
6.1.2 Testcase A: Gaussian low-pass filter -> image bluring



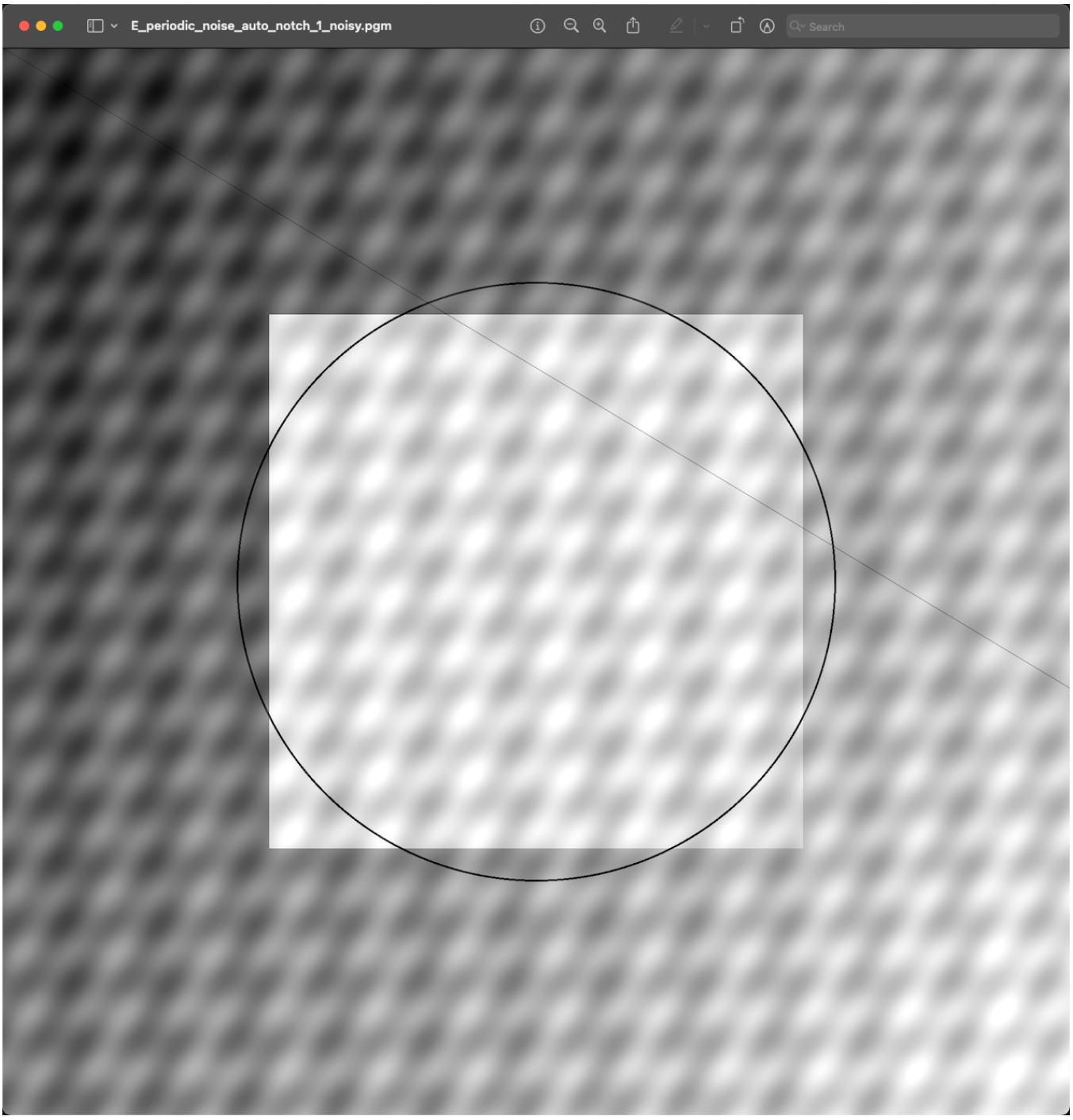
6.1.3 Testcase B: Gaussian high-pass filter -> Edge Detection



6.1.4 Testcase C: bandpass filter -> Texture Extraction

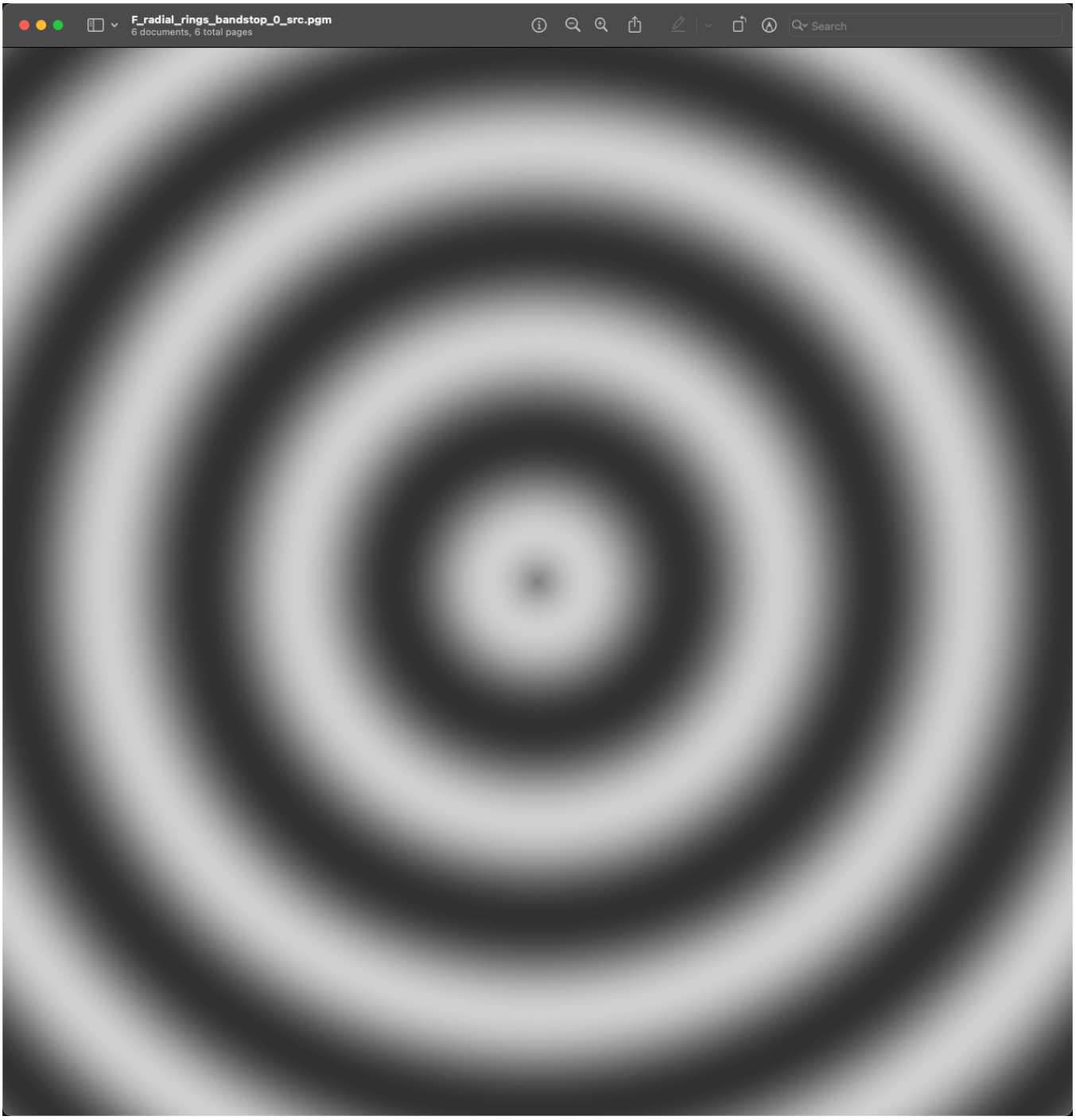


6.1.5 Testcase E: Periodic noise -> auto notch denoise (auto only)

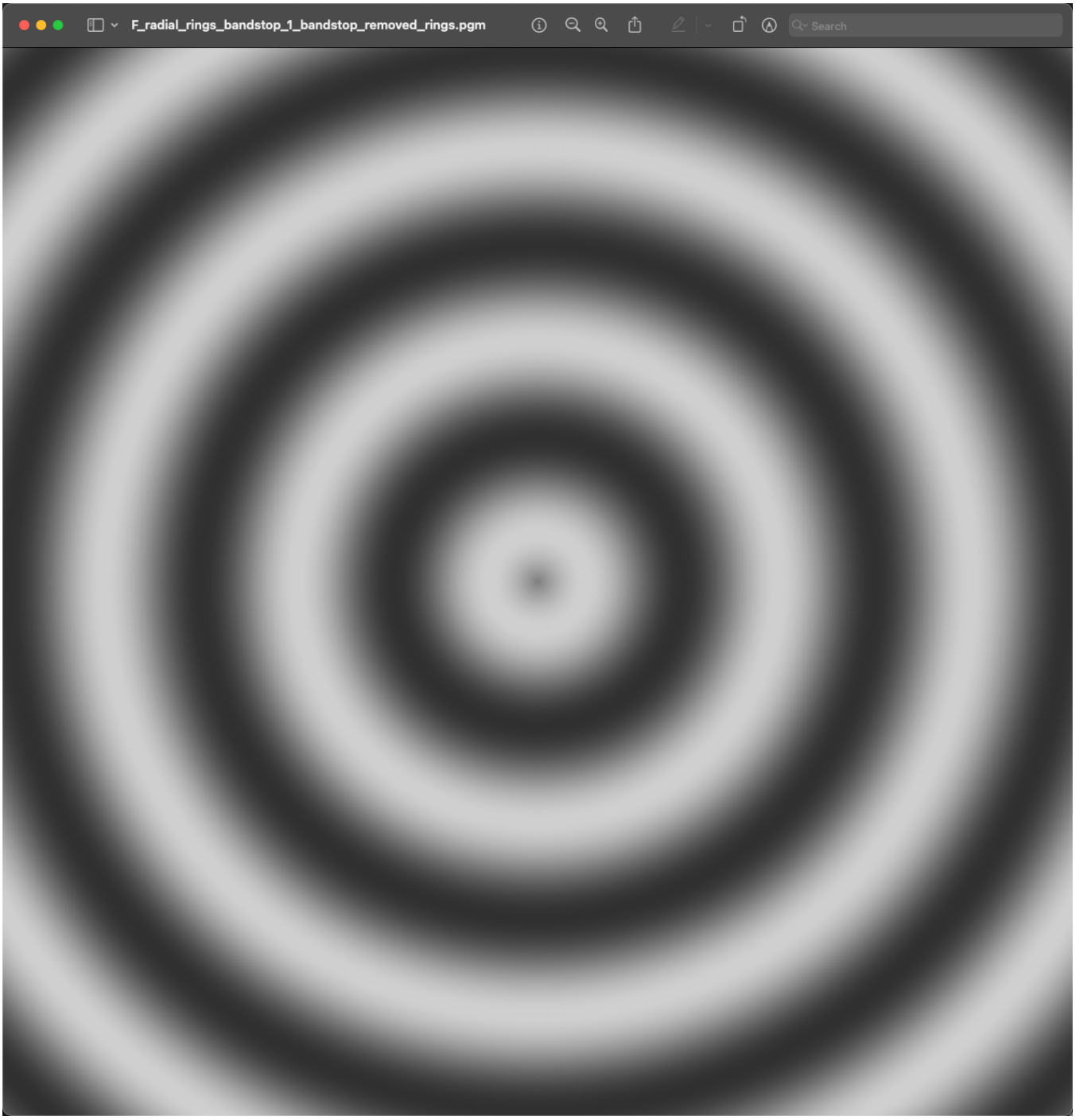


6.1.6 Testcase F: band-stop filter -> Radial rings

orginale image



after filtering

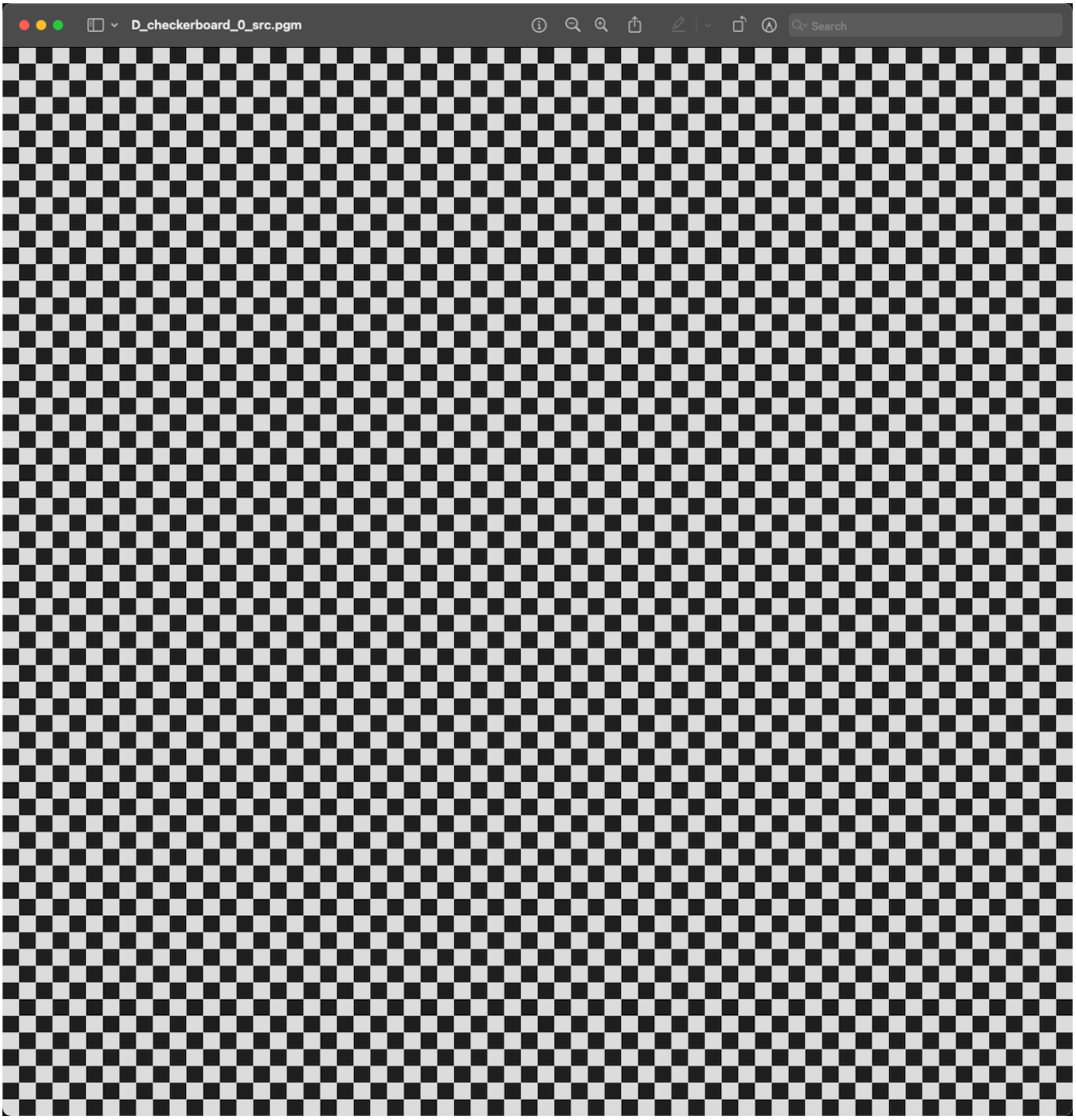


6.1.7 Testcase D: Checkerboard stress

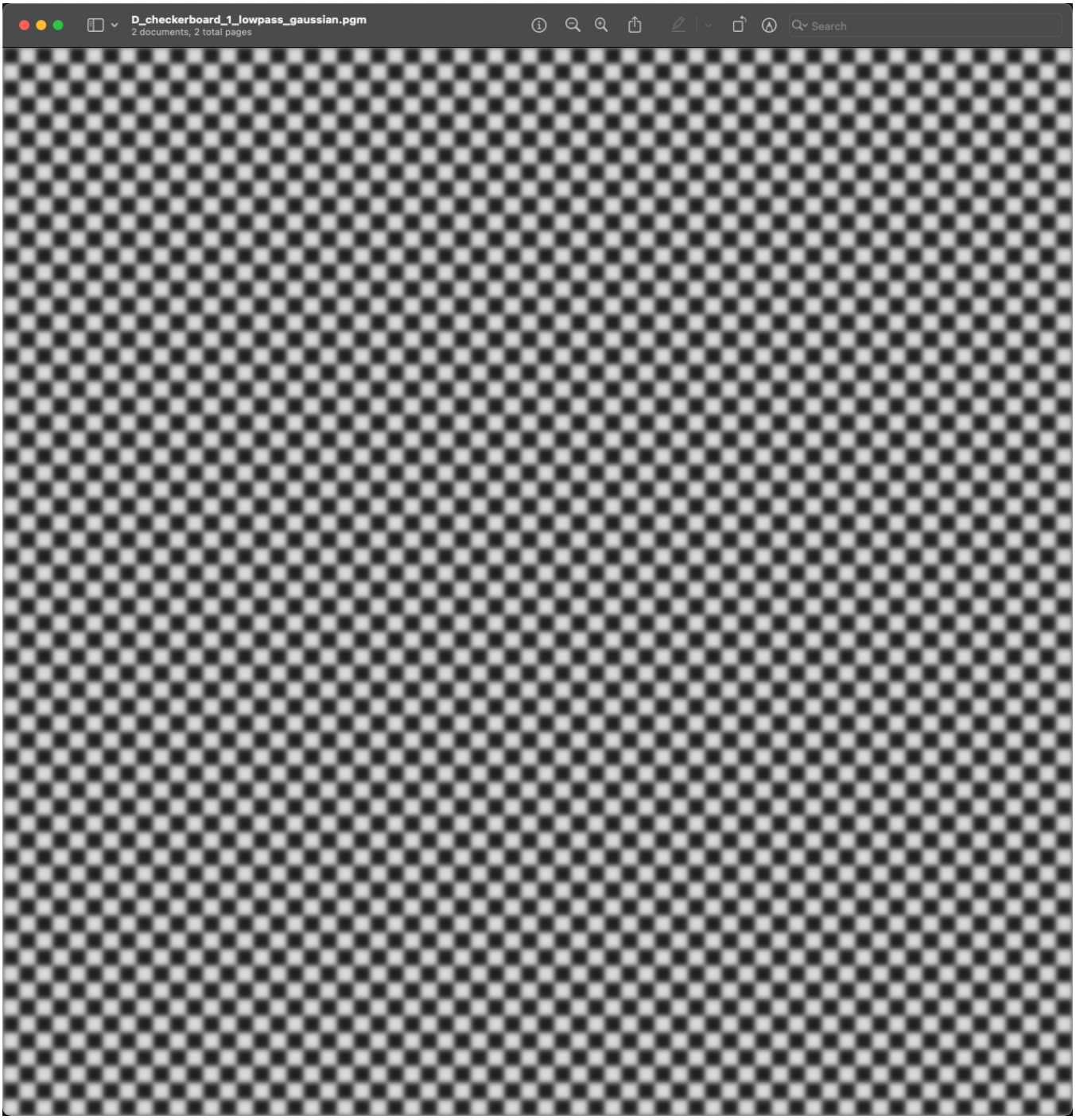
it:

- Generates a checkerboard image (high-frequency pattern).
- Computes its 2D DFT.
- Applies a Gaussian low-pass filter in the frequency domain.
- Performs the inverse DFT to reconstruct the filtered image.
- Saves images, spectra, timing results, and quality metrics.

original image



filtered image



6.2 metrics and performance

6.2.1 dft_cpu

metrics.csv

```
test,N,name,ref,hash_u8,hash_f64,l2,rmse,max_abs,psnr
A_gauss_lowpass_blur,2048,A_gauss_lowpass_blur_0_src,,10866791419739912012,18265540612519131329,0,0,0,0
A_gauss_lowpass_blur,2048,A_gauss_lowpass_blur_0_src_spectrum.pgm,,3749688621329275414,5621963975890653079,0,
0,0,0
SPECTRUM,2048,A_gauss_lowpass_blur_0_src_spectrum_fromF.pgm,,3749688621329275414,5621963975890653079,0,0,0,0
A_gauss_lowpass_blur,2048,A_gauss_lowpass_blur_1_blur,A_gauss_lowpass_blur_0_src,3456739244970335134,11958356
224658597048,18365.6,8.96757,196.501,29.0773
A_gauss_lowpass_blur,2048,A_gauss_lowpass_blur_1_blur_spectrum.pgm,,17560898503544143786,6612922541910154349,
0,0,0,0
B_gauss_highpass_edges,2048,B_gauss_highpass_edges_0_src,,10866791419739912012,18265540612519131329,0,0,0,0
B_gauss_highpass_edges,2048,B_gauss_highpass_edges_0_src_spectrum.pgm,,3749688621329275414,5621963975890653079
,0,0,0,0
SPECTRUM,2048,B_gauss_highpass_edges_0_src_spectrum_fromF.pgm,,3749688621329275414,5621963975890653079,0,0,0,
0
B_gauss_highpass_edges,2048,B_gauss_highpass_edges_1_edges_norm,,13918824126237060822,511564752789105787,0,0,
```

```
0,0
B_gauss_highpass_edges,2048,B_gauss_highpass_edges_1_edges_norm_spectrum.pgm,,7149525401828900317,11184706116
458556519,0,0,0,0
C_bandpass_texture,2048,C_bandpass_texture_0_src,,10866791419739912012,18265540612519131329,0,0,0,0
C_bandpass_texture,2048,C_bandpass_texture_0_src_spectrum.pgm,,3749688621329275414,5621963975890653079,0,0,0,
0
SPECTRUM,2048,C_bandpass_texture_0_src_spectrum_fromF.pgm,,3749688621329275414,5621963975890653079,0,0,0,0
C_bandpass_texture,2048,C_bandpass_texture_1_bandpass_norm,,9818206463593958077,15973326002350086089,0,0,0,0
C_bandpass_texture,2048,C_bandpass_texture_1_bandpass_norm_spectrum.pgm,,3195924986459601118,8525137401638388
867,0,0,0,0
D_checkerboard,2048,D_checkerboard_0_src,,14861834530626609957,14220677151323202341,0,0,0,0
D_checkerboard,2048,D_checkerboard_0_src_spectrum.pgm,,13953749360887124954,12772053891648620531,0,0,0,0
SPECTRUM,2048,D_checkerboard_0_src_spectrum_fromF.pgm,,13953749360887124954,12772053891648620531,0,0,0,0
D_checkerboard,2048,D_checkerboard_1_lowpass_gaussian,D_checkerboard_0_src,15466597265096221477,3861498868935
194931,108035,52.7517,94.4628,13.6861
D_checkerboard,2048,D_checkerboard_1_lowpass_gaussian_spectrum.pgm,,12165870305972638682,10550330624670537784
,0,0,0,0
E_periodic_noise_auto_notch,2048,E_periodic_noise_auto_notch_0_base_clean,,10866791419739912012,1826554061251
9131329,0,0,0,0
E_periodic_noise_auto_notch,2048,E_periodic_noise_auto_notch_0_base_clean_spectrum.pgm,,3749688621329275414,5
621963975890653079,0,0,0,0
E_periodic_noise_auto_notch,2048,E_periodic_noise_auto_notch_1_noisy,E_periodic_noise_auto_notch_0_base_clean
,8246501991637648432,10504034471513538023,37467.9,18.2949,44.9859,22.8842
E_periodic_noise_auto_notch,2048,E_periodic_noise_auto_notch_1_noisy_spectrum.pgm,,9768228011621837558,281824
2349200602511,0,0,0,0
SPECTRUM,2048,E_periodic_noise_auto_notch_1_noisy_spectrum_fromF.pgm,,9768228011621837558,2818242349200602511
,0,0,0,0
E_periodic_noise_auto_notch,2048,E_periodic_noise_auto_notch_2_denoised_auto_notch,E_periodic_noise_auto_notc
h_0_base_clean,4077599198027618096,5804308800435205393,38319.1,18.7105,244.944,22.6891
E_periodic_noise_auto_notch,2048,E_periodic_noise_auto_notch_2_denoised_auto_notch_spectrum.pgm,,497746885565
9379431,9924773373197751415,0,0,0,0
F_radial_rings_bandstop,2048,F_radial_rings_bandstop_0_src,,12729338138157003600,10259394259384313912,0,0,0,0
F_radial_rings_bandstop,2048,F_radial_rings_bandstop_0_src_spectrum.pgm,,1793776746216188088,1657260018829337
0122,0,0,0,0
SPECTRUM,2048,F_radial_rings_bandstop_0_src_spectrum_fromF.pgm,,1793776746216188088,16572600188293370122,0,0,
0,0
F_radial_rings_bandstop,2048,F_radial_rings_bandstop_1_bandstop_removed_rings,F_radial_rings_bandstop_0_src,3
054508027361530248,13049889087619014843,87.0804,0.0425197,1.1745,75.559
F_radial_rings_bandstop,2048,F_radial_rings_bandstop_1_bandstop_removed_rings_spectrum.pgm,,58499084322028486
88,923933604586104208,0,0,0,0
```

performance.csv

```
test,N,stage,ms
A_gauss_lowpass_blur,2048,DFT2,31506.9
A_gauss_lowpass_blur,2048,Mask_gaussian_lowpass,55.388
A_gauss_lowpass_blur,2048,IDFT2,32296.6
B_gauss_highpass_edges,2048,DFT2,32036.9
B_gauss_highpass_edges,2048,Mask_gaussian_highpass,54.1183
B_gauss_highpass_edges,2048,IDFT2,31036.9
C_bandpass_texture,2048,DFT2,31059.6
C_bandpass_texture,2048,Mask_bandpass_ideal,54.2328
C_bandpass_texture,2048,IDFT2,31355.5
D_checkerboard,2048,DFT2,31861
D_checkerboard,2048,Mask_gaussian_lowpass,56.6244
D_checkerboard,2048,IDFT2,32200.7
E_periodic_noise_auto_notch,2048,DFT2,31868.1
E_periodic_noise_auto_notch,2048,Build_auto_notch,198.554
E_periodic_noise_auto_notch,2048,Mask_auto_notch,56.0465
E_periodic_noise_auto_notch,2048,IDFT2,32041
F_radial_rings_bandstop,2048,DFT2,31335.6
F_radial_rings_bandstop,2048,Mask_bandstop_ideal,56.0247
F_radial_rings_bandstop,2048,IDFT2,31627.2
```

6.2.2 fft_cpu

metrics.csv

```
test,N,name,ref,hash_u8,hash_f64,l2,rmse,max_abs,psnr
A_gauss_lowpass_blur,2048,A_gauss_lowpass_blur_0_src,,10866791419739912012,18265540612519131329,0,0,0,0
A_gauss_lowpass_blur,2048,A_gauss_lowpass_blur_0_src_spectrum.pgm,,3749688621329275414,14837453148438810876,0
,0,0,0
SPECTRUM,2048,A_gauss_lowpass_blur_0_src_spectrum_fromF.pgm,,3749688621329275414,14837453148438810876,0,0,0,0
A_gauss_lowpass_blur,2048,A_gauss_lowpass_blur_1_blur,A_gauss_lowpass_blur_0_src,3456739244970335134,54570927
3469566195,18365.6,8.96757,196.501,29.0773
A_gauss_lowpass_blur,2048,A_gauss_lowpass_blur_1_blur_spectrum.pgm,,17560898503544143786,6835529153870673989,
0,0,0,0
B_gauss_highpass_edges,2048,B_gauss_highpass_edges_0_src,,10866791419739912012,18265540612519131329,0,0,0,0
B_gauss_highpass_edges,2048,B_gauss_highpass_edges_0_src_spectrum.pgm,,3749688621329275414,148374531484388108
76,0,0,0,0
SPECTRUM,2048,B_gauss_highpass_edges_0_src_spectrum_fromF.pgm,,3749688621329275414,14837453148438810876,0,0,0
,0
B_gauss_highpass_edges,2048,B_gauss_highpass_edges_1_edges_norm,,13918824126237060822,11078132632299104543,0,
0,0,0
B_gauss_highpass_edges,2048,B_gauss_highpass_edges_1_edges_norm_spectrum.pgm,,7149525401828900317,61986308610
63257712,0,0,0,0
C_bandpass_texture,2048,C_bandpass_texture_0_src,,10866791419739912012,18265540612519131329,0,0,0,0
C_bandpass_texture,2048,C_bandpass_texture_0_src_spectrum.pgm,,3749688621329275414,14837453148438810876,0,0,0
,0
SPECTRUM,2048,C_bandpass_texture_0_src_spectrum_fromF.pgm,,3749688621329275414,14837453148438810876,0,0,0,0
C_bandpass_texture,2048,C_bandpass_texture_1_bandpass_norm,,9818206463593958077,6538876893288647229,0,0,0,0
C_bandpass_texture,2048,C_bandpass_texture_1_bandpass_norm_spectrum.pgm,,3195924986459601118,9831809110311206
888,0,0,0,0
D_checkerboard,2048,D_checkerboard_0_src,,14861834530626609957,14220677151323202341,0,0,0,0
D_checkerboard,2048,D_checkerboard_0_src_spectrum.pgm,,13953749360887124954,4166399302478962576,0,0,0,0
SPECTRUM,2048,D_checkerboard_0_src_spectrum_fromF.pgm,,13953749360887124954,4166399302478962576,0,0,0,0
D_checkerboard,2048,D_checkerboard_1_lowpass_gaussian,D_checkerboard_0_src,15466597265096221477,1841335113528
4540197,108035,52.7517,94.4628,13.6861
D_checkerboard,2048,D_checkerboard_1_lowpass_gaussian_spectrum.pgm,,12165870305972638682,2834872759762353500,
0,0,0,0
E_periodic_noise_auto_notch,2048,E_periodic_noise_auto_notch_0_base_clean,,10866791419739912012,1826554061251
9131329,0,0,0,0
E_periodic_noise_auto_notch,2048,E_periodic_noise_auto_notch_0_base_clean_spectrum.pgm,,3749688621329275414,1
48374531484388108 76,0,0,0,0
E_periodic_noise_auto_notch,2048,E_periodic_noise_auto_notch_1_noisy,E_periodic_noise_auto_notch_0_base_clean
,8246501991637648432,10504034471513538023,37467.9,18.2949,44.9859,22.8842
E_periodic_noise_auto_notch,2048,E_periodic_noise_auto_notch_1_noisy_spectrum.pgm,,9768228011621837558,850379
0110506157614,0,0,0,0
SPECTRUM,2048,E_periodic_noise_auto_notch_1_noisy_spectrum_fromF.pgm,,9768228011621837558,8503790110506157614
,0,0,0,0
E_periodic_noise_auto_notch,2048,E_periodic_noise_auto_notch_2_denoised_auto_notch,E_periodic_noise_auto_notc
h_0_base_clean,4077599198027618096,11472745299070604976,38319.1,18.7105,244.944,22.6891
E_periodic_noise_auto_notch,2048,E_periodic_noise_auto_notch_2_denoised_auto_notch_spectrum.pgm,,497746885565
9379431,14732971787415803861,0,0,0,0
F_radial_rings_bandstop,2048,F_radial_rings_bandstop_0_src,,12729338138157003600,10259394259384313912,0,0,0,0
F_radial_rings_bandstop,2048,F_radial_rings_bandstop_0_src_spectrum.pgm,,1793776746216188088,8430708386132480
179,0,0,0,0
SPECTRUM,2048,F_radial_rings_bandstop_0_src_spectrum_fromF.pgm,,1793776746216188088,8430708386132480179,0,0,0
,0
F_radial_rings_bandstop,2048,F_radial_rings_bandstop_1_bandstop_removed_rings,F_radial_rings_bandstop_0_src,3
0545080273615302 48,16862932158991852137,87.0804,0.0425197,1.1745,75.559
F_radial_rings_bandstop,2048,F_radial_rings_bandstop_1_bandstop_removed_rings_spectrum.pgm,,584990843220284866
88,4871590872415211273,0,0,0,0
```

performance.csv

```
test,N,stage,ms
A_gauss_lowpass_blur,2048,FFT2,291.479
A_gauss_lowpass_blur,2048,Mask_gaussian_lowpass,54.5366
A_gauss_lowpass_blur,2048,IFFT2,338.55
B_gauss_highpass_edges,2048,FFT2,294.991
B_gauss_highpass_edges,2048,Mask_gaussian_highpass,53.6698
B_gauss_highpass_edges,2048,IFFT2,312.774
C_bandpass_texture,2048,FFT2,295.69
C_bandpass_texture,2048,Mask_bandpass_ideal,56.0918
C_bandpass_texture,2048,IFFT2,338.357
```

```
D_checkerboard,2048,FFT2,289.839
D_checkerboard,2048,Mask_gaussian_lowpass,55.3141
D_checkerboard,2048,IFFT2,340.945
E_periodic_noise_auto_notch,2048,FFT2,279.766
E_periodic_noise_auto_notch,2048,Build_auto_notch,199.678
E_periodic_noise_auto_notch,2048,Mask_auto_notch,54.6902
E_periodic_noise_auto_notch,2048,IFFT2,334.445
F_radial_rings_bandstop,2048,FFT2,288.079
F_radial_rings_bandstop,2048,Mask_bandstop_ideal,54.9513
F_radial_rings_bandstop,2048,IFFT2,340.26
```

6.2.3 dft_gpu

metrics.csv

```
test,N,name,ref,hash_u8,hash_f64,l2,rmse,max_abs,psnr
A_gauss_lowpass_blur,2048,A_gauss_lowpass_blur_0_src,,10866791419739912012,18265540612519131329,0,0,0,0
A_gauss_lowpass_blur,2048,A_gauss_lowpass_blur_0_src_spectrum.pgm,,3749688621329275414,8963780625368898420,0,
0,0,0
SPECTRUM,2048,A_gauss_lowpass_blur_0_src_spectrum_fromF.pgm,,3749688621329275414,8963780625368898420,0,0,0,0
A_gauss_lowpass_blur,2048,A_gauss_lowpass_blur_1_blur,A_gauss_lowpass_blur_0_src,3456739244970335134,14917904
166632397283,18365.6,8.96757,196.501,29.0773
A_gauss_lowpass_blur,2048,A_gauss_lowpass_blur_1_blur_spectrum.pgm,,17560898503544143786,7891143390803246666,
0,0,0,0
B_gauss_highpass_edges,2048,B_gauss_highpass_edges_0_src,,10866791419739912012,18265540612519131329,0,0,0,0
B_gauss_highpass_edges,2048,B_gauss_highpass_edges_0_src_spectrum.pgm,,3749688621329275414,8963780625368898420,0,0,0,0
SPECTRUM,2048,B_gauss_highpass_edges_0_src_spectrum_fromF.pgm,,3749688621329275414,8963780625368898420,0,0,0,
0
B_gauss_highpass_edges,2048,B_gauss_highpass_edges_1_edges_norm,,13918824126237060822,9659583521362895826,0,0
,0,0
B_gauss_highpass_edges,2048,B_gauss_highpass_edges_1_edges_norm_spectrum.pgm,,7149525401828900317,10237002225
186817354,0,0,0,0
C_bandpass_texture,2048,C_bandpass_texture_0_src,,10866791419739912012,18265540612519131329,0,0,0,0
C_bandpass_texture,2048,C_bandpass_texture_0_src_spectrum.pgm,,3749688621329275414,8963780625368898420,0,0,0,
0
SPECTRUM,2048,C_bandpass_texture_0_src_spectrum_fromF.pgm,,3749688621329275414,8963780625368898420,0,0,0,0
C_bandpass_texture,2048,C_bandpass_texture_1_bandpass_norm,,9818206463593958077,1092274550134550077,0,0,0,0
C_bandpass_texture,2048,C_bandpass_texture_1_bandpass_norm_spectrum.pgm,,3195924986459601118,4095765222938023
612,0,0,0,0
D_checkerboard,2048,D_checkerboard_0_src,,14861834530626609957,14220677151323202341,0,0,0,0
D_checkerboard,2048,D_checkerboard_0_src_spectrum.pgm,,13953749360887124954,491051422541973898,0,0,0,0
SPECTRUM,2048,D_checkerboard_0_src_spectrum_fromF.pgm,,13953749360887124954,491051422541973898,0,0,0,0
D_checkerboard,2048,D_checkerboard_1_lowpass_gaussian,D_checkerboard_0_src,15466597265096221477,1231698034514
1664220,108035,52.7517,94.4628,13.6861
D_checkerboard,2048,D_checkerboard_1_lowpass_gaussian_spectrum.pgm,,12165870305972638682,17316460478101949264
,0,0,0,0
E_periodic_noise_auto_notch,2048,E_periodic_noise_auto_notch_0_base_clean,,10866791419739912012,1826554061251
9131329,0,0,0,0
E_periodic_noise_auto_notch,2048,E_periodic_noise_auto_notch_0_base_clean_spectrum.pgm,,3749688621329275414,8
963780625368898420,0,0,0,0
E_periodic_noise_auto_notch,2048,E_periodic_noise_auto_notch_1_noisy,E_periodic_noise_auto_notch_0_base_clean
,8246501991637648432,10504034471513538023,37467.9,18.2949,44.9859,22.8842
E_periodic_noise_auto_notch,2048,E_periodic_noise_auto_notch_1_noisy_spectrum.pgm,,9768228011621837558,379113
4243653324788,0,0,0,0
SPECTRUM,2048,E_periodic_noise_auto_notch_1_noisy_spectrum_fromF.pgm,,9768228011621837558,3791134243653324788
,0,0,0,0
E_periodic_noise_auto_notch,2048,E_periodic_noise_auto_notch_2_denoised_auto_notch,E_periodic_noise_auto_notc
h_0_base_clean,4077599198027618096,14155585285766299115,38319.1,18.7105,244.944,22.6891
E_periodic_noise_auto_notch,2048,E_periodic_noise_auto_notch_2_denoised_auto_notch_spectrum.pgm,,497746885565
9379431,14621882955000253641,0,0,0,0
F_radial_rings_bandstop,2048,F_radial_rings_bandstop_0_src,,12729338138157003600,10259394259384313912,0,0,0,0
F_radial_rings_bandstop,2048,F_radial_rings_bandstop_0_src_spectrum.pgm,,1793776746216188088,1460006488406063
831,0,0,0,0
SPECTRUM,2048,F_radial_rings_bandstop_0_src_spectrum_fromF.pgm,,1793776746216188088,1460006488406063831,0,0,0
,0
F_radial_rings_bandstop,2048,F_radial_rings_bandstop_1_bandstop_removed_rings,F_radial_rings_bandstop_0_src,3
054508027361530248,17211933558168081251,87.0804,0.0425197,1.1745,75.559
F_radial_rings_bandstop,2048,F_radial_rings_bandstop_1_bandstop_removed_rings_spectrum.pgm,,58499084322028486
```

```
88,13835337105317591167,0,0,0,0
```

performance.csv

```
test,N,stage,ms
A_gauss_lowpass_blur,2048,DFT2,911.656
A_gauss_lowpass_blur,2048,Mask_gaussian_lowpass,5.80218
A_gauss_lowpass_blur,2048,IDFT2,906.58
B_gauss_highpass_edges,2048,DFT2,913.626
B_gauss_highpass_edges,2048,Mask_gaussian_highpass,5.01339
B_gauss_highpass_edges,2048,IDFT2,908.79
C_bandpass_texture,2048,DFT2,914.296
C_bandpass_texture,2048,Mask_bandpass_ideal,4.56169
C_bandpass_texture,2048,IDFT2,908.942
D_checkerboard,2048,DFT2,914.167
D_checkerboard,2048,Mask_gaussian_lowpass,5.24713
D_checkerboard,2048,IDFT2,909.083
E_periodic_noise_auto_notch,2048,DFT2,914.698
E_periodic_noise_auto_notch,2048,Build_auto_notch,196.728
E_periodic_noise_auto_notch,2048,Mask_auto_notch,5.33553
E_periodic_noise_auto_notch,2048,IDFT2,909.092
F_radial_rings_bandstop,2048,DFT2,915.219
F_radial_rings_bandstop,2048,Mask_bandstop_ideal,4.54964
F_radial_rings_bandstop,2048,IDFT2,909.108
```

6.2.4 fft_gpu

metrics.csv

```
test,N,name,ref,hash_u8,hash_f64,l2,rmse,max_abs,psnr
A_gauss_lowpass_blur,2048,A_gauss_lowpass_blur_0_src,,10866791419739912012,18265540612519131329,0,0,0,0
A_gauss_lowpass_blur,2048,A_gauss_lowpass_blur_0_src_spectrum.pgm,,3749688621329275414,17892655024533849139,0
,0,0,0
SPECTRUM,2048,A_gauss_lowpass_blur_0_src_spectrum_fromF.pgm,,3749688621329275414,17892655024533849139,0,0,0,0
A_gauss_lowpass_blur,2048,A_gauss_lowpass_blur_1_blur,A_gauss_lowpass_blur_0_src,3456739244970335134,14738067
456634233350,18365.6,8.96757,196.501,29.0773
A_gauss_lowpass_blur,2048,A_gauss_lowpass_blur_1_blur_spectrum.pgm,,17560898503544143786,14916870382666200968
,0,0,0,0
B_gauss_highpass_edges,2048,B_gauss_highpass_edges_0_src,,10866791419739912012,18265540612519131329,0,0,0,0
B_gauss_highpass_edges,2048,B_gauss_highpass_edges_0_src_spectrum.pgm,,3749688621329275414,178926550245338491
39,0,0,0,0
SPECTRUM,2048,B_gauss_highpass_edges_0_src_spectrum_fromF.pgm,,3749688621329275414,17892655024533849139,0,0,0
,0
B_gauss_highpass_edges,2048,B_gauss_highpass_edges_1_edges_norm,,13918824126237060822,11320739330674247538,0,
0,0,0
B_gauss_highpass_edges,2048,B_gauss_highpass_edges_1_edges_norm_spectrum.pgm,,7149525401828900317,55448713514
1810567,0,0,0,0
C_bandpass_texture,2048,C_bandpass_texture_0_src,,10866791419739912012,18265540612519131329,0,0,0,0
C_bandpass_texture,2048,C_bandpass_texture_0_src_spectrum.pgm,,3749688621329275414,17892655024533849139,0,0,0
,0
SPECTRUM,2048,C_bandpass_texture_0_src_spectrum_fromF.pgm,,3749688621329275414,17892655024533849139,0,0,0,0
C_bandpass_texture,2048,C_bandpass_texture_1_bandpass_norm,,9818206463593958077,12375129866696058200,0,0,0,0
C_bandpass_texture,2048,C_bandpass_texture_1_bandpass_norm_spectrum.pgm,,3195924986459601118,5079172676004104
610,0,0,0,0
D_checkerboard,2048,D_checkerboard_0_src,,14861834530626609957,14220677151323202341,0,0,0,0
D_checkerboard,2048,D_checkerboard_0_src_spectrum.pgm,,13953749360887124954,10837382192346723676,0,0,0,0
SPECTRUM,2048,D_checkerboard_0_src_spectrum_fromF.pgm,,13953749360887124954,10837382192346723676,0,0,0,0
D_checkerboard,2048,D_checkerboard_1_lowpass_gaussian,D_checkerboard_0_src,15466597265096221477,3310209770417
640229,108035,52.7517,94.4628,13.6861
D_checkerboard,2048,D_checkerboard_1_lowpass_gaussian_spectrum.pgm,,12165870305972638682,15047345822322085881
,0,0,0,0
E_periodic_noise_auto_notch,2048,E_periodic_noise_auto_notch_0_base_clean,,10866791419739912012,1826554061251
9131329,0,0,0,0
E_periodic_noise_auto_notch,2048,E_periodic_noise_auto_notch_0_base_clean_spectrum.pgm,,3749688621329275414,1
7892655024533849139,0,0,0,0
E_periodic_noise_auto_notch,2048,E_periodic_noise_auto_notch_1_noisy,E_periodic_noise_auto_notch_0_base_clean
,8246501991637648432,10504034471513538023,37467.9,18.2949,44.9859,22.8842
E_periodic_noise_auto_notch,2048,E_periodic_noise_auto_notch_1_noisy_spectrum.pgm,,9768228011621837558,152344
```

```
32573942360974,0,0,0,0
SPECTRUM,2048,E_periodic_noise_auto_notch_1_noisy_spectrum_fromF.pgm,,9768228011621837558,1523443257394236097
4,0,0,0,0
E_periodic_noise_auto_notch,2048,E_periodic_noise_auto_notch_2_denoised_auto_notch,E_periodic_noise_auto_notc
h_0_base_clean,4077599198027618096,5523555151506843096,38319.1,18.7105,244.944,22.6891
E_periodic_noise_auto_notch,2048,E_periodic_noise_auto_notch_2_denoised_auto_notch_spectrum.pgm,,497746885565
9379431,6896660963920066905,0,0,0,0
F_radial_rings_bandstop,2048,F_radial_rings_bandstop_0_src,,12729338138157003600,10259394259384313912,0,0,0,0
F_radial_rings_bandstop,2048,F_radial_rings_bandstop_0_src_spectrum.pgm,,1793776746216188088,1146402763712800
7618,0,0,0,0
SPECTRUM,2048,F_radial_rings_bandstop_0_src_spectrum_fromF.pgm,,1793776746216188088,11464027637128007618,0,0,
0,0
F_radial_rings_bandstop,2048,F_radial_rings_bandstop_1_bandstop_removed_rings,F_radial_rings_bandstop_0_src,3
0545080273615302 48,13093019960871532893,87.0804,0.0425197,1.1745,75.559
F_radial_rings_bandstop,2048,F_radial_rings_bandstop_1_bandstop_removed_rings_spectrum.pgm,,58499084322028486
88,9031577996335884755,0,0,0,0
```

performance.csv

```
test,N,stage,ms
A_gauss_lowpass_blur,2048,FFT2,19.9224
A_gauss_lowpass_blur,2048,Mask_gaussian_lowpass,4.96598
A_gauss_lowpass_blur,2048,IFFT2,15.1765
B_gauss_highpass_edges,2048,FFT2,20.4252
B_gauss_highpass_edges,2048,Mask_gaussian_highpass,4.58151
B_gauss_highpass_edges,2048,IFFT2,15.17
C_bandpass_texture,2048,FFT2,20.3382
C_bandpass_texture,2048,Mask_bandpass_ideal,4.61456
C_bandpass_texture,2048,IFFT2,15.1633
D_checkerboard,2048,FFT2,20.3242
D_checkerboard,2048,Mask_gaussian_lowpass,5.05914
D_checkerboard,2048,IFFT2,15.154
E_periodic_noise_auto_notch,2048,FFT2,20.0705
E_periodic_noise_auto_notch,2048,Build_auto_notch,195.498
E_periodic_noise_auto_notch,2048,Mask_auto_notch,4.74061
E_periodic_noise_auto_notch,2048,IFFT2,15.172
F_radial_rings_bandstop,2048,FFT2,20.4185
F_radial_rings_bandstop,2048,Mask_bandstop_ideal,4.15549
F_radial_rings_bandstop,2048,IFFT2,15.1636
```

6.3 Performance comparison

total time in ms :

algorithm execution time (ms)	A: lowpass	B: highpass	C: bandpass	D: Checkerboard Stress	E: Auto-notch denoise	F: bandstop
DFT in cpu	108014.7776	114964.3412	115416.3875	110225.1407	113494.5547	111946.6822
FFT in cpu	684.5656	661.4348	690.1388	686.0981	868.5792	683.2903
DFT in gpu	1824.0382	1827.4294	1827.7997	1828.4971	2025.8535	1828.8766
FFT in gpu	40.0649	40.1767	40.1161	40.5373	235.4811	39.7376

Algorithm Comparison	A: lowpass	B: highpass	C: bandpass	D: Checkerboard Stress	E: Auto-notch denoise	F: bandstop
DFTcpu_vs_FFTcpu	0.9937	0.9942	0.9940	0.9938	0.9923	0.9939
DFTcpu_vs_DFTgpu	0.9831	0.9841	0.9842	0.9834	0.9822	0.9837
FFTcpu_vs_FFTgpu	0.9415	0.9393	0.9419	0.9409	0.7289	0.9418
DFTcpu_vs_FFTgpu	0.9996	0.9997	0.9997	0.9996	0.9979	0.9996

- First row: the absolute value of (DFT in CPU minus FFT in CPU) divided by DFT in CPU.
- Second row: the absolute value of (DFT in CPU minus DFT in GPU) divided by DFT in CPU.
- Third row: the absolute value of (FFT in CPU minus FFT in GPU) divided by FFT in CPU.
- Fourth row: the absolute value of (DFT in CPU minus FFT in GPU) divided by DFT in CPU.

Algorithm Comparison	**A: lowpass**	**B: highpass**	**C: bandpass**	**D: Checkerboard Stress**	**E: Auto-notch denoise**	**F: bandstop**
abs(DFTcpu-FFTcpu)/FFTcpu	156.7859	172.8105	166.2365	159.6551	129.6669	162.8347
abs(DFTcpu-DFTgpu)/DFTgpu	58.2174	61.9104	62.1450	59.2818	55.0231	60.2106
abs(FFTcpu-FFTgpu)/FFTgpu	16.0864	15.4631	16.2035	15.9251	2.6885	16.1951
abs(DFTcpu-FFTgpu)/FFTgpu	2694.9952	2860.4680	2876.0590	2718.1042	480.9688	2816.1475

- The first row represents the absolute value of DFT on CPU minus FFT on CPU, divided by FFT on CPU.
- The second row represents the absolute value of DFT on CPU minus DFT on GPU, divided by DFT on GPU.
- The third row represents the absolute value of FFT on CPU minus FFT on GPU, divided by FFT on GPU.
- The fourth row represents the absolute value of DFT on CPU minus FFT on GPU, divided by FFT on GPU.